WayDrop
Supabase Operations Handbook

Production Administration, Admin Operations & Troubleshooting Manual

Version 1.0 — Generated 2026-08-26 from repository source (migrations, Edge Functions, Flutter client) and hand-maintained project docs.

No live production database credentials were available at generation time (no service-role key, no linked Supabase CLI session). Schema, RLS, triggers, functions and Edge Function logic are verified directly against repository source, which is authoritative for structure. Live row VALUES (e.g. today's exact subscription price, current production_config flag states) could only be sourced from migration defaults/seed data and are explicitly labeled "NOT VERIFIED IN CURRENT PROJECT" wherever a Supabase Dashboard edit since deploy could have changed them.

This handbook is a read-only documentation reference. It was generated by inspecting the repository and (where noted) hand-maintained docs; no production data was inserted, updated, or deleted, no migration or Edge Function was deployed, and no secret was read or changed while producing it. Wherever a fact could not be independently verified against actual source code, it is explicitly marked NOT VERIFIED rather than assumed.

Known documentation caveat: several of the project's hand-maintained docs (docs/01, 08, 09, 10, 12, 13, 14, 15, 16) were found during preparation of this handbook to be a templated batch with some drift from current source (stale table/bucket names, unresolved numeric-constant conflicts). Every fact drawn from those docs is cross-checked against actual migrations/Edge Function source in this handbook and flagged if it could not be confirmed. docs/07_Payments.md was independently confirmed reliable and code-verified throughout.

Table of Contents

Table of Contents 2

PART I: QUICK START 8

Chapter 1. How to Use This Handbook . . . 8

Callout conventions used throughout . . . 8

Chapter 2. Supabase Project Overview & Source-of-Truth Rules . . . 9

Chapter 3. Production Safety Rules . . . 10

Chapter 4. Emergency Controls Quick Reference . . . 10

Chapter 5. "Something Is Broken — What Do I Check First?" . . . 11

PART II: SUPABASE ARCHITECTURE 13

Chapter 6. Supabase Project Architecture . . . 13

Data flow for a transactional write . . . 13

Storage . . . 13

Auth . . . 14

Chapter 7. Auth, Users, Riders & Partners . . . 14

Chapter 8. Database Schema Overview . . . 14

Chapter 9. Table Relationship Map . . . 15

Chapter 10. RLS and Security Model . . . 16

Chapter 11. Triggers and Database Functions . . . 17

Chapter 12. Edge Functions Overview . . . 17

Chapter 13. Realtime . . . 18

Chapter 14. Storage Buckets . . . 18

Chapter 15. Scheduled Jobs & Background Loops . . . 19

Chapter 16. Production Configuration . . . 19

PART III: WAYDROP BUSINESS MODULES 21

Chapter 17. Local Ride Booking . . . 21

Chapter 18. Local Parcel . . . 21

Chapter 19. Intercity Ride & Parcel Matching . . . 21

Chapter 20. Partner/Rider Online System & Dispatch . . . 22

Chapter 21. Active Rides & Order Lifecycle . . . 22

Chapter 22. Ride Completion, Cancellation & Settlement . . . 23
Chapter 23. Notifications . . . 23
Chapter 24. Ratings and Feedback . . . 24
Chapter 25. Subscriptions (WayDrop MAX) & Free Trial . . . 24
Chapter 26. Vehicle-Specific Pricing . . . 25
Chapter 27. Payments & Cashfree . . . 25
Chapter 28. Government Tax . . . 26
Chapter 29. Payouts & Withdrawals . . . 26
Chapter 30. KYC and Documents . . . 27
Chapter 31. Partner Vehicles . . . 27
Chapter 32. User/Partner Profiles & Account Deletion . . . 27

PART IV: ADMIN OPERATIONS 28

Chapter 33. Feature Kill-Switches . . . 28
Maintenance Mode . . . 28
Force Update / Minimum & Latest Android Version . . . 28
Disable / Enable Local Rides, Parcel, Intercity . . . 29
Disable / Enable Payments & Subscriptions . . . 29
Disable / Enable Notifications . . . 30
Chapter 34. Admin Read-Only Inspection Recipes . . . 30
Viewing active partners (riders) . . . 30
Viewing active rides . . . 30
Viewing orders stuck in searching . . . 31
Viewing intercity requests . . . 31
Viewing KYC documents and photos . . . 31
Viewing withdrawals and payout history . . . 31
Viewing payments . . . 32
Viewing subscriptions . . . 32
Viewing ratings . . . 32
Viewing notifications and failed notifications . . . 32
Viewing payment failures . . . 33
Chapter 35. Investigating Stuck Orders . . . 33
Chapter 36. Changing Subscription Prices . . . 33

PART V: TROUBLESHOOTING 35

Chapter 37. Local Ride Booking Not Working . . . 35

Chapter 38. Local Parcel Not Working . . . 35
Chapter 39. Intercity Matching Not Working . . . 35
Chapter 40. Partner Cannot Go Online / Cannot Accept Ride . . . 36
Chapter 41. Ride Stuck in Searching / Stuck After Acceptance . . . 37
Chapter 42. Payment Not Working / Cashfree Webhook Problems . . . 37
Chapter 43. Subscription Not Activating / Price Wrong / Free Trial Issues . . . 38
Chapter 44. KYC Documents / Photos Not Showing . . . 39
Chapter 45. Notification Not Arriving / Tap Does Nothing . . . 39
Chapter 46. Rating Feedback Not Showing . . . 40
Chapter 47. Withdrawal Not Working . . . 40
Chapter 48. Cashfree Webhook Problem . . . 41
Chapter 49. Government Tax Problem . . . 41
Chapter 50. Realtime Problem . . . 41
Chapter 51. App Appears Slow . . . 41
Chapter 52. Duplicate Requests / Duplicate Notifications . . . 42
Chapter 53. Production Emergency Response . . . 42

PART VI: DATABASE REFERENCE 43

Chapter 54. Complete Table Directory . . . 43
Chapter 55. Core Identity & Account Tables . . . 45
users . . . 45
rider_kyc . . . 45
riders . . . 45
rider_vehicles . . . 46
Chapter 56. Order & Dispatch Tables . . . 46
orders . . . 46
rider_assignments / order_assignment_logs . . . 46
order_status_history . . . 47
Chapter 57. Intercity Tables . . . 47
rider_travel_plans . . . 47
intercity_requests . . . 47
travel_matches . . . 47
Chapter 58. Payments & Financial Tables . . . 47
payments . . . 47
subscription_plans . . . 48
rider_subscription_history . . . 48

government_tax_config / tax_rate_config / government_tax_ledger . . . 48
cod_settlement_ledger / wallet_transactions / wallet_audit_logs . . . 48
Chapter 59. Payout & Withdrawal Tables . . . 48
rider_payout_accounts / rider_payout_updates_log . . . 48
rider_withdrawals / rider_withdrawal_events . . . 48
Chapter 60. Notification Tables . . . 49
notifications . . . 49
notification_failures . . . 49
Chapter 61. Ratings, Config & Supporting Tables . . . 49
rider_reviews . . . 49
production_config, dispatch_config, pricing_config, pricing_zones . . . 49
promo_codes / promo_code_users / user_promo_usages . . . 49
Compliance & safety tables . . . 49
Account lifecycle tables . . . 50

PART VII: FUNCTION & EDGE FUNCTION REFERENCE 51

Chapter 62. SQL Function Directory . . . 51
Chapter 63. Trigger Directory . . . 52
Chapter 64. Edge Function Directory Overview . . . 52
Chapter 65. create-order . . . 53
Chapter 66. accept-order . . . 54
Chapter 67. accept-intercity-request . . . 54
Chapter 68. update-order-status . . . 54
Chapter 69. start-dispatch . . . 54
Chapter 70. match-intercity-riders . . . 55
Chapter 71. create-payment . . . 55
Chapter 72. cashfree-webhook & verify-payment . . . 55
Chapter 73. create-fare-qr & boost-order-fare . . . 55
Chapter 74. send-push-notification . . . 56
Chapter 75. regenerate-otp . . . 56
Chapter 76. create-rider-kyc, validate-rider-kyc, process-verification-photo, bypass-verification-photo 56
Chapter 77. create-rider-payout & create-rider-withdrawal . . . 56
Chapter 78. delete-account & cleanup-abandoned-kyc-uploads . . . 57
Chapter 79. Other Edge Functions . . . 57

PART VIII: EMERGENCY PLAYBOOK 58

Chapter 80. Feature Kill-Switch Quick Reference . . . 58
Chapter 81. Maintenance Mode & Force Update . . . 58
Chapter 82. Incident Response Checklist . . . 58
Chapter 83. Rollback Guidance . . . 59

PART IX: APP / SUPABASE CROSS-REFERENCE 60

Chapter 84. Flutter Screen → Supabase Object Map . . . 60
Chapter 85. Flutter Service → Edge Function Map . . . 61
Chapter 86. Table → Function Map . . . 61
Chapter 87. Notification Type → Producer → Recipient . . . 62
Chapter 88. Payment Flow → Tables → Functions . . . 62
Chapter 89. Subscription Flow → Tables → Functions . . . 63
Chapter 90. KYC Flow → Storage → Tables → Screens . . . 63

PART X: ADMIN CHEAT SHEETS 64

Chapter 91. 1-Page Emergency Cheat Sheet . . . 64
Chapter 92. Subscription Price Change Cheat Sheet . . . 64
Chapter 93. KYC Cheat Sheet . . . 64
Chapter 94. Active Ride Cheat Sheet . . . 65
Chapter 95. Withdrawal & Payment Cheat Sheet . . . 65
Chapter 96. Notification & Feature-Disable Cheat Sheet . . . 65

PART XI: APPENDICES & CHANGE LOG 66

Chapter 97. Appendix A — What Requires What (Decision Framework) . . . 66
Chapter 98. Appendix B — Known Project-Specific Safety Rules . . . 66
Chapter 99. Appendix C — Release Checklist (composed from `docs/11_Deployment.md` and this handbook's own findings) . . . 67
Chapter 100. Appendix D — Cashfree Phase 7B Deferred Checklist . . . 68
Chapter 101. Appendix E — Change Log . . . 68

Notifications & Realtime . . . 68
Subscriptions . . . 68
Performance . . . 68
Payments architecture . . . 69
KYC . . . 69
Production stability batches . . . 69
Intercity / UX . . . 69

Chapter 102. Appendix F — How This Handbook Was Produced 69

Alphabetical Index **69**

PART I

QUICK START

Chapter 1. How to Use This Handbook

This handbook documents the WayDrop production system as it actually exists in the repository at generation time: the Supabase Postgres schema (tables, RLS, triggers, functions), the 28 Deno Edge Functions, the Flutter client's admin-relevant surfaces, and the operational knowledge captured in project docs, migration comments, commit history, and regression-test headers.

It is organized so that a specific question ("why is this rider's withdrawal stuck?", "how do I change the bike subscription price?", "what does disabling intercity actually do to a trip in progress?") can be answered by going to one place, rather than re-reading the whole document.

- **Part II — Architecture**: how the pieces fit together (Auth, Realtime, RLS, triggers, Edge Functions, Storage, production_config).
- **Part III — Business Modules**: how each WayDrop feature actually works end-to-end.
- **Part IV — Admin Operations**: how to change something safely, and how to look something up.
- **Part V — Troubleshooting**: symptom-first chapters using a consistent template.
- **Part VI/VII — Database and Function Reference**: the full table-by-table and function-by-function catalog.
- **Part VIII — Emergency Playbook**: the condensed kill-switch/incident quick reference.
- **Part IX — Cross-Reference**: Flutter ↔ Supabase mapping.
- **Part X — Cheat Sheets**: one page per common task.
- **Part XI — Appendices & Change Log**.
- **Part XII — Alphabetical Index**, generated last so page numbers are exact.

Callout conventions used throughout

WARNING
Something that can cause real damage (revenue, data integrity, stranded rides) if done carelessly.

SAFE
An operation that is low-risk and reversible.

READ-ONLY
A SELECT-only query — safe to run against production for diagnosis.

PRODUCTION
A note specifically about live production behavior/state.

APK REQUIRED
This change requires a new Android/iOS app build and store release to take effect for users.

EDGE FUNCTION DEPLOY REQUIRED
This change requires `supabase functions deploy <name>` to take effect.

MIGRATION REQUIRED
This change requires a new forward-only SQL migration (`supabase migration new`, then `supabase db push`).

A "NOT VERIFIED IN CURRENT PROJECT" tag means: this handbook's authors could not confirm the exact current value/behavior against a live production system (no database credentials were available), only against repository source. Treat these as "true as of the last deploy of this code", not necessarily "true right now."

Chapter 2. Supabase Project Overview & Source-of-Truth Rules

- Linked project ref (from `supabase/.temp/project-ref`): **yvwpchffctiagevpqgwy**.
- The repository's `supabase/migrations/` directory (248 files at generation time) is the authoritative, forward-only definition of the database schema. Migrations are never edited after being merged — a correction is always a new migration.
- `supabase/functions/<name>/index.ts` is the authoritative source for Edge Function behavior. There is no Node tooling/package.json — everything is Deno.
- The Flutter client under `lib/` is authoritative for what end-users and Drop Partners (riders) actually see and can do.
- **This handbook was generated without live Supabase database credentials** (`.env` in this repository defines only `SUPABASE_URL`, `SUPABASE_ANON_KEY`, `OLA_MAPS_API_KEY` — no service-role key, no DB password; no `supabase` CLI was installed/linked in the generation environment). Every "current value" claim (subscription prices, production_config flags, tax rates) is therefore sourced from migration defaults/seed data, which is the value *as last deployed* — an admin may have changed it since directly in the Supabase Dashboard, which for several of these tables (subscription_plans, production_config) is the explicitly documented, supported way to make the change. Always re-confirm the live value in the Supabase Table Editor before treating a number in this handbook as current truth.

WARNING — Docs vs. code
Several of this project's own docs (`docs/01`, `08`, `09`, `10`, `12`, `13`, `14`, `15`, `16_*.md`) were found to be a single templated batch (identical "Version 1.0, July 5 2026" metadata) with confirmed drift from actual source — stale storage bucket names, edge functions listed that need re-verification, and two conflicting numeric constants for the same dispatch-radius setting within the same document. `docs/07_Payments.md` was independently verified reliable throughout. This handbook cross-checks every fact against actual migrations/Edge Function source rather than trusting the docs at face value, and says so inline where a doc's claim could not be confirmed.

Chapter 3. Production Safety Rules

DO	DON'T
• Read `production_config`, `subscription_plans`, and other tables via the Supabase Table Editor or a SELECT query to diagnose an issue. • Change `subscription_plans.amount` or a `production_config` boolean directly via the Supabase Dashboard/SQL editor — this is the documented, supported mechanism for both. • Use the SECURITY DEFINER RPC functions (`initiate_withdrawal`, `complete_withdrawal`, `fail_withdrawal`, `apply_order_government_tax`, etc.) when a manual financial correction is genuinely required. • Preserve every accepted/ongoing ride when using an emergency kill-switch — the flags are new-action gates, not lifecycle interrupts.	• Directly UPDATE `orders.order_status`, any fare/commission column, `riders.wallet_balance`, `riders.total_earnings`, or `riders.government_tax_due_balance` — these are protected by trigger and/or only correctly mutated by specific SECURITY DEFINER functions; a raw UPDATE breaks ledger consistency. • Delete a `rider_kyc` row, a `rider_reviews` row, or any row from an append-only ledger (`government_tax_ledger`, `rider_withdrawal_events`, `notification_failures`) — these exist specifically for audit/compliance. • Resend or bulk-reprocess old `notifications` rows, or treat `notification_failures` as a retry queue — no retry mechanism exists by design. • Deploy an Edge Function, push a migration, or flip a production_config flag without explicit approval if you are operating under a documented approval-gate process for this project.

This handbook's own generation followed the same rule: it performed read-only inspection only, and made no production changes.

Chapter 4. Emergency Controls Quick Reference

Full detail is in **Part IV, Chapter — Feature Kill-Switches** and **Part VIII — Emergency Playbook**. This is the one-glance version.

Flag (production_config column)	Effect when FALSE / TRUE flips it off	Strands in-progress trips?
maintenance_mode	TRUE blocks the entire app (even logged-out users) behind a maintenance screen, checked before login.	No — the block happens only at app cold-start routing, before a session resumes an in-progress order screen state; it does not touch an order's server-side lifecycle.
force_update	TRUE blocks app usage below `minimum_android_version` with no bypass.	No — same as above, a client-side gate only.
local_rides_enabled / ride_booking_enabled	FALSE greys out the "Ride" tile client-side AND makes `create-order`/`accept-order` return 503 for new ride/ride_sharing orders server-side.	No — `update-order-status` (everything after acceptance) is intentionally never gated by any of these flags.
parcel_enabled	FALSE greys out the "Parcel" tile and blocks new `local_parcel` orders server-side (create-order/accept-order).	No.

Flag (production_config column)	Effect when FALSE / TRUE flips it off	Strands in-progress trips?
intercity_enabled	FALSE greys out the "Intercity" tile and blocks new intercity_ride/intercity_parcel orders server-side (create-order/accept-order/ accept-intercity-request).	No.
payments_enabled	Master kill switch for ALL new Cashfree payment sessions (order fares, subscriptions, tax dues, COD settlement) via `create-payment`. No client-side UI toggle exists — the server's 503 message IS the UX.	No — an order already past payment collection is unaffected; only new payment-session creation is blocked.
subscriptions_enabled	Required in addition to `payments_enabled` specifically for `payment_for='subscription'`.	No.
notifications_enabled	Stops new outbound FCM pushes via `send-push-notification`. Does NOT touch the in-app notification inbox — `notifications` rows are still inserted, only the push delivery is skipped.	No — never affects the order/business lifecycle.

WARNING — Fail-open by design

Every one of these server-side checks reads a 30-second in-memory cache of `production_config` and is explicitly **fail-open**: if the config row can't be fetched (outage, network blip), the check defaults to "enabled", never "disabled". An outage in the config system itself can never accidentally strand bookings, payments, or notifications — it can only ever fail to enforce a restriction you intended to have in place. Confirm the flag actually took effect by testing the specific blocked action, not just by trusting that the row was updated.

PRODUCTION

There is no live-push mechanism to already-open client sessions. A flag change takes effect for a given user's app on their next cold start (`AppStartScreen.checkUserFlow()`), which is driven by app launch / auth-state change, not a mid-session poll. A user already deep inside the app when you flip a flag will not be interrupted mid-session.

Chapter 5. "Something Is Broken — What Do I Check First?"

Use this as a triage funnel before jumping into a specific Part V chapter.

1. **Is it one user/rider, or everyone?** One user → look at that user's/order's/rider's rows directly (Part IV, Admin Read-Only Inspection Recipes). Everyone / a whole feature → check `production_config` first (Part IV, Feature Kill-Switches) — someone may have intentionally or accidentally disabled a flag.
2. **Is it a NEW action (booking, accepting, paying, subscribing) or something ALREADY in progress (an accepted ride, an ongoing chat)?** New actions are the only things production_config flags ever gate. An in-progress ride breaking is never a feature-flag issue — look at `update-order-status`, Realtime, or the specific lifecycle chapter in Part III instead.
3. **Is data simply not showing, or is an action actively failing/erroring?** Not showing → check Realtime connectivity (Part V, Realtime Problem) or a stale client cache before assuming the data is wrong server-side. Actively erroring → get the exact error code/message; most Edge Functions here return a structured `{success:false, code, message}` body that maps directly to a specific cause (see

the function's entry in Part VII).

4. **Did this used to work?** If a specific date is known, check the changelog in the Appendix and `git log` around that date — several fixes in this codebase were themselves fixing a previous regression.
5. **Is money involved (a payment, a withdrawal, a subscription, tax due)?** Go straight to the relevant Part V chapter — these have the most SECURITY DEFINER-function-only correction paths and the highest cost of a wrong manual fix.
6. When in doubt, read the specific table's or function's entry in Part VI/VII before writing any SQL — several columns here are trigger-protected and a raw UPDATE will silently violate an invariant rather than error.

PART II

SUPABASE ARCHITECTURE

Chapter 6. Supabase Project Architecture

WayDrop uses three distinct client-server channels, each for a different kind of data:

1. **REST via Edge Functions** (Deno, `service_role`, bypasses RLS) — every transactional operation: order creation, pricing, acceptance, payments, KYC, withdrawals. The client calls `supabase.functions.invoke('<name>')` with its own user JWT; the function verifies the JWT, applies business rules the client cannot be trusted to enforce itself (fare amount, subscription duration, order ownership), and writes via the `service_role` key.
2. **Realtime** (Postgres logical replication / WebSocket) — a single multiplexed WebSocket subscribed to filtered Postgres changes (`orders` by id, `order_chat_messages` by order_id, `riders` by geofence) for live UI updates without polling.
3. **FCM push** — for background/killed-app delivery, driven server-side by a DB trigger → `pg_net` HTTP call → the `send-push-notification` Edge Function → Firebase Cloud Messaging.

Data flow for a transactional write

```
Flutter client

  -> supabase.functions.invoke('create-order', { Authorization: Bearer <user JWT> })

  -> Edge Function verifies JWT via supabase.auth.getUser(token)

  -> Edge Function re-derives/validates everything money- or state-related server-side

  -> Edge Function writes via a service_role Supabase client (bypasses RLS)

  -> Postgres trigger(s) fire (e.g. notification insert, dispatch trigger)

  -> pg_net fires an async HTTP call to another Edge Function if needed (e.g. push)

  -> JSON response returns to the client
```

> **WARNING — Never trust the client for money or state**
> This phrase appears literally in code comments across `create-order`, `create-payment`, `accept-order`, and `update-order-status`. Every fare amount, subscription duration, tax amount, and order-status transition is re-derived or re-validated server-side from the client's JWT-verified identity and from server-held rows — never taken as-is from the request body. A confirmed historical vulnerability (see the Subscriptions chapter and the Appendix Change Log) existed precisely because one code path once violated this rule.

Storage

Three Supabase Storage buckets are confirmed from migrations/code in this pass: `rider_kyc` (private since 2026-07-31, KYC identity documents), `driver_personal_qr` (private, a rider's own uploaded UPI QR image for direct passenger payment), and `parcel_images` (parcel photos and pickup/dropoff verification photos). A `docs/01_Architecture.md` reference to a `kyc_docs` bucket is confirmed stale documentation — the real bucket has always been `rider_kyc`.

Auth

Supabase Phone OTP → JWT → Row-Level Security keyed off `auth.uid()`. OTP delivery is NOT Supabase's built-in provider — a `sms-hook` Edge Function intercepts every OTP Supabase Auth wants to send and routes it through MSG91 (an India SMS provider) instead, authenticated via Supabase's `standardwebhooks` HMAC scheme rather than a JWT.

Chapter 7. Auth, Users, Riders & Partners

There is no separate "partners" table. The single `riders` table is the Drop Partner account record; a passenger is a row in `users`. A single phone number can hold both a `users` row and, if they onboard as a Drop Partner, a linked `riders` row (dual-mode), tracked via `users.preferred_mode`/`last_active_mode`.

Concept	Table	Notes
Passenger account	`users`	PK = `auth.users.id`. Holds notification preferences, `role` (user/admin/support), `order_notification_mode`.
Drop Partner (rider) account	`riders`	FK to `rider_kyc` and `users`. Holds rating, earnings, wallet_balance, live location, account_status (online/offline/busy/blocked), subscription state.
Rider identity/KYC	`rider_kyc`	Aadhaar/PAN/DL numbers and 10 document images, kyc_status, activation_status.
Admin/support account	`admins`	Referenced by `production_config.updated_by_admin_id` and `admin_logs`; gates write RLS on config-style tables per `docs/10_Admin.md` (NOT independently re-verified against migrations in this pass — treat the exact RLS mechanism as NOT VERIFIED IN CURRENT PROJECT until confirmed against the actual `admins`-table migration).

WARNING — role escalation is trigger-guarded

`users.role` is protected by a `prevent_role_self_escalation` trigger — a user cannot grant themselves admin/support via a normal client write path. Granting admin/support access is an intentionally out-of-band operation; do not attempt it via a client-facing update path.

There is no in-app admin panel for rider onboarding decisions in this codebase (confirmed by repo-wide grep during a prior security audit, cited in migration comments). Rider activation, KYC approval/rejection, and admin broadcast notifications are Supabase Dashboard-only operations today — see Part IV.

Chapter 8. Database Schema Overview

The schema is defined entirely by 248 forward-only migrations under `supabase/migrations/`, containing roughly 55 tables, 118 SQL functions, and 46 triggers at generation time. The full table directory is in Part VI; this section gives the shape of the schema by domain.

Domain	Core tables
Identity & KYC	users, riders, rider_kyc, rider_vehicles, admins
Local order lifecycle	orders, order_assignment_logs, order_status_history, rider_assignments, order_verifications
Intercity	rider_travel_plans, intercity_requests, travel_matches
Payments & money	payments, subscription_plans, rider_subscription_history, wallet_transactions, wallet_audit_logs, cod_settlement_ledger, payment_disputes
Government tax	tax_rate_config, government_tax_config, government_tax_ledger
Payouts	rider_payout_accounts, rider_payout_updates_log, rider_withdrawals, rider_withdrawal_events
Notifications	notifications, notification_failures, user_device_tokens
Ratings	rider_reviews
Pricing & dispatch config	pricing_zones, pricing_config, pricing_change_logs, dispatch_config, distance_discounts
Compliance & safety	order_declarations, order_parcel_photos, parcel_security_logs, parcel_compliance_reports, police_sos_logs, otp_attempts, trip_share_tokens
Promotions	promo_codes, promo_code_users, user_promo_usages
Account lifecycle	data_deletion_requests, deleted_user_archive, rider_kyc_retention_archive
Operational config	production_config, admin_logs, diagnostic_timings (temporary)

WARNING — There is no `intercity_orders` table, no `partners` table, no `rider_subscriptions` table
These are natural-sounding names that do NOT exist in this schema and should not be assumed by anyone extending this handbook or writing new SQL. Use `intercity_requests` + `orders` + `rider_travel_plans` for intercity, `riders` for partners, and `rider_subscription_history` for subscription state.

Chapter 9. Table Relationship Map

Key foreign-key relationships (arrows read "references"):

```
users --(1:0..1)--> riders (a user who onboards as a Drop Partner)

riders --(1:1)--> rider_kyc

riders --(1:N)--> rider_vehicles

riders --(1:N)--> rider_travel_plans

riders --(1:1)--> rider_payout_accounts

riders --(1:N)--> rider_withdrawals

orders --(N:1)--> users            (passenger)

orders --(N:1)--> riders           (assigned rider, nullable until accepted)

orders --(N:1)--> pricing_config
```

```
orders --(1:0..1)--> payments

orders --(1:N)--> order_status_history

orders --(1:N)--> rider_assignments

orders --(1:0..1)--> intercity_requests   (intercity orders only)

intercity_requests --(N:1)--> rider_travel_plans

intercity_requests --(N:1)--> riders      (traveller)

intercity_requests --(N:1)--> users       (sender)

payments --(N:1)--> subscription_plans    (subscription payments only)

payments --(N:0..1)--> riders             (subscription/withdrawal-adjacent payments)

rider_withdrawals --(N:1)--> riders

rider_withdrawals --(1:N)--> rider_withdrawal_events

rider_reviews --(1:1)--> orders (UNIQUE — one review per order)

rider_reviews --(N:1)--> riders

government_tax_ledger --(N:1)--> orders / payments (per entry_type)

notification_failures --(N:0..1)--> notifications (ON DELETE SET NULL)
```

Chapter 10. RLS and Security Model

Row-Level Security is enabled on every user-facing table; the general pattern is "a row is visible/writable only to the `auth.uid()` that owns it," with three notable categories of exception:

- **Config/reference tables** (`production_config`, `subscription_plans`) allow broad `SELECT` (including `anon` for `production_config`, so a logged-out client can still see maintenance/force-update state) but have **no INSERT/UPDATE/DELETE policy for anon or authenticated at all** — writes are default-deny, meaning only `service_role` or a project owner acting through the Supabase Dashboard/SQL editor can change them. This is a deliberate, documented design, not an oversight.
- **Financial mutation paths** (`rider_withdrawals`, `government_tax_ledger`, wallet-adjacent columns on `riders`) similarly have no client-facing write policy — every legitimate write goes through a SECURITY DEFINER RPC function, which runs with elevated privilege internally regardless of the caller's own RLS.
- **Two-party visibility** (`rider_reviews`, `intercity_requests`) — visible to both participants (the reviewer and the rated rider; the sender and the traveller) but no one else, and in `rider_reviews`'s case, immutable after INSERT for everyone including admins via any client-facing policy.

> **WARNING — A real historical regression**
> A previous, insecure "public read" RLS policy on `rider_kyc` was found and removed (migrations `20260808000007` and `20260808000008`). Treat the discovery of a broad/public SELECT policy on any identity or financial table as a regression to fix immediately, not as an intentional design choice, unless a migration comment explicitly justifies it.

A second, easy-to-miss layer exists alongside RLS: several tables in this project also require an explicit Postgres `GRANT` to the `authenticated`/`anon` role in addition to a permissive RLS policy — RLS policies alone are not sufficient, and PostgREST will return "permission denied" without the grant even if the policy

would otherwise allow the row.

Chapter 11. Triggers and Database Functions

118 SQL functions and 46 triggers exist across the schema. The heaviest-used categories, with representative functions:

Category	Representative functions
Dispatch/matching	`find_nearby_riders`, `assign_nearby_searching_orders_to_rider`, `redispatch_freed_riders`, `expire_stale_searching_orders`
Order lifecycle	`generate_unique_order_code`, `complete_order_and_settle`, `credit_rider_for_completed_order`, `calculate_fare_breakdown`
Intercity	`accept_intercity_request`, `cancel_intercity_request_by_traveller`, `on_intercity_request_change`, `sync_travel_plan_status_after`
Notifications	`send_push_notification_fnc`, `deliver_push_notification_http_fnc`, `on_order_status_update`, `on_new_chat_message`, `on_rider_review_insert`
Payments/tax/wallet	`apply_order_government_tax`, `apply_government_tax_payment`, `is_rider_blocked_by_tax_due`, `activate_rider_free_trial`
Withdrawals/payouts	`initiate_withdrawal`, `fail_withdrawal`, `complete_withdrawal`, `generate_rider_beneficiary_id`
Security/RLS helpers	`check_is_order_passenger`, `prevent_role_self_escalation`, `protect_rider_financial_columns`

> **SAFE**
> PL/pgSQL triggers in this codebase never make external HTTP calls directly — they queue work into a table (typically `notifications`) and let an asynchronous step (`pg_net`, then an Edge Function) process it. This is a hard project convention (see `docs/12_Troubleshooting.md` and the entire notification pipeline design) — a trigger blocking on a slow external call would hold a database lock or transaction open far longer than acceptable.

A single generic `set_updated_at` trigger function is reused across most tables that carry an `updated_at` column, rather than each table having its own bespoke version.

The full alphabetical function and trigger directory is in Part VII.

Chapter 12. Edge Functions Overview

28 Deno Edge Functions exist under `supabase/functions/`, plus a `_shared/` directory of cross-cutting helpers used by many of them:

Shared helper	Purpose
`http.ts`	`fetchWithTimeout()` wraps every outbound fetch (Ola Maps, Cashfree, FCM, function-to-function) with a default 15-second AbortController timeout, so a slow peer produces a catchable error instead of hanging the whole invocation indefinitely.
`http_errors.ts`	Standard `HttpError` class so business-logic failures (invalid transition, conflict, invalid OTP) return the correct HTTP status instead of a generic 500.
`production_config_cache.ts`	30-second in-memory cache of the single active `production_config` row; fail-open (see Part I). This is the server-side enforcement half of every feature flag.
`dispatch_config_cache.ts`, `pricing_config_cache.ts`, `eta_cache.ts`, `signed_url_cache.ts`, `ola_client.ts`	Same TTL-cache pattern applied to dispatch tuning, pricing rules, ETA lookups, signed Storage URLs, and a shared Ola Maps client.

Full per-function detail (purpose, auth model, tables touched, feature-flag enforcement, Flutter callers) is in Part VII.

Chapter 13. Realtime

WayDrop maintains a single multiplexed Realtime WebSocket subscribing to filtered Postgres changes on `orders` (by id, for order tracking), `order_chat_messages` (by order_id, for in-order chat), and `riders` (within a geofence, for live rider positions). This depends on `REPLICA IDENTITY FULL` being set on the watched tables (`users`, `riders`, `orders`, `payments` all have it) so that Realtime can deliver both old and new row values on UPDATE.

> **PRODUCTION**
> Commit `e1c0f8e` ("harden realtime pooling and cache cleanup") is a real, recent change to this subsystem — any exact reconnect-backoff timings or heartbeat intervals documented elsewhere should be re-confirmed against the current `lib/core/services/realtime/` source rather than assumed stable, since this handbook did not re-verify those exact numbers against post-fix source in this pass.

If order/matching updates stop arriving live: check the Realtime connection state first (Part V, Realtime Problem), then confirm the affected table still has `REPLICA IDENTITY FULL` and is still part of the `supabase_realtime` publication — removing a table from that publication silently stops Realtime delivery without an error anywhere else in the stack.

Chapter 14. Storage Buckets

Bucket	Visibility	Contents	Path convention
`rider_kyc`	Private since 2026-07-31 (previously public — a fixed P1 security issue)	10 KYC images per rider (Aadhaar front/back, PAN front/back, DL front/back, vehicle RC front/back, rider photo, vehicle photo)	`{user_id}/{phone}_attempt_{n}/{fileName}.jpg`
`driver_personal_qr`	Private	A rider's own uploaded UPI QR code image, shown fullscreen to a passenger for direct payment	`{rider_id}/...`

Bucket	Visibility	Contents	Path convention
`parcel_images`	Public (URLs embedded directly in `orders.parcel_images`/`order_parcel_photos`)	Parcel photos taken at booking and pickup/dropoff verification photos	`images/<orderCode>/<timestamp>_<i>.<ext>`

WARNING — Only two of ten KYC images are ever shown in the app

Of the 10 images stored per rider in the `rider_kyc` bucket, only `rider_image.jpg` and `vehicle_image.jpg` are ever rendered client-side (confirmed by a repo-wide grep during a prior security audit). The other eight (Aadhaar/PAN/DL/RC) are KYC-review-only and are never displayed in the Flutter app itself — reviewing them today means opening the Supabase Storage browser directly (see Part IV, KYC Cheat Sheet).

Since the `rider_kyc` bucket is private, every legitimate read site resolves a fresh signed URL on demand rather than storing a permanent public URL — `create-rider-kyc` stores bare storage paths, not full URLs, specifically so this keeps working after the bucket's visibility changed.

Chapter 15. Scheduled Jobs & Background Loops

There is no dedicated pg_cron chapter's worth of confirmed jobs in this pass beyond what is directly evidenced in code comments: a scheduled sweep of `cleanup-abandoned-kyc-uploads` (removing orphaned KYC upload sessions), and a documented "1-minute pg_cron backstop" that also calls `expire_stale_searching_orders()` (the same idempotent function `start-dispatch` calls itself when a search gives up) — this backstop's exact schedule/definition was not independently located in this pass and should be confirmed directly in the Supabase Dashboard's Cron section or via `SELECT * FROM cron.job;` before being relied upon. Treat any other scheduled job as NOT VERIFIED IN CURRENT PROJECT until confirmed the same way.

READ-ONLY

```
-- List all scheduled jobs on the linked Supabase project

SELECT jobid, schedule, command, active FROM cron.job;
```

Chapter 16. Production Configuration

`production_config` (migration `20260901000034_production_config.sql`) is the current, canonical live feature-flag and emergency-control table. It replaced an earlier `feature_flags` table that was deliberately dropped (migration `20260702000000_drop_feature_flags.sql`, "remote feature flags are no longer used") — that old table and its Flutter counterpart `lib/core/services/session/feature_flag_service.dart` are dead code today and must not be resurrected or confused with `production_config`.

Column	Type	Default (as migrated)	Purpose
`config_code`	text	`'default'`, unique	Lets multiple config rows exist; only the one with `is_active = true` is live.
`is_active`	boolean	—	Exactly one row may be true at a time (enforced by a partial unique index). This is how an admin swaps configs without deleting history.

Column	Type	Default (as migrated)	Purpose
`minimum_android_version`	text	`'1.0.0'`	Below this app version, the client is force-blocked. Compared against the app's version string, not its build number.
`latest_android_version`	text	`'1.0.0'`	Shown in non-blocking "update available" messaging.
`force_update`	boolean	`false`	Hard-blocks app usage until updated, independent of the version comparison.
`update_message`	text	generic update text	Shown on the force/soft update screen.
`maintenance_mode`	boolean	`false`	Blocks the entire app behind a maintenance screen.
`maintenance_message`	text	generic maintenance text	Shown on the maintenance screen.
`local_rides_enabled`, `ride_booking_enabled`, `intercity_enabled`, `parcel_enabled`, `payments_enabled`, `subscriptions_enabled`, `notifications_enabled`	boolean	`true`	Per-module kill switches — see the Quick Reference table in Part I and the full recipes in Part IV.
`extra_flags`	jsonb	`{}`	Escape hatch for a future simple flag without a new migration.
`updated_by_admin_id`	uuid, FK → admins	NULL	Audit trail of which admin last changed the config.

All values above are migration defaults; a Dashboard edit since deploy is the documented way live values change, so re-confirm the live row before treating any of these as current — see Part IV for the exact read query.

RLS: `anon` and `authenticated` may `SELECT` only the row where `is_active = true`; there is no INSERT/UPDATE/DELETE policy for either role, so the Supabase Dashboard/SQL editor (as project owner, or `service_role`) is the only path to change it. A single `set_updated_at` trigger keeps `updated_at` current.

PART III

WAYDROP BUSINESS MODULES

Chapter 17. Local Ride Booking

A local ride is `orders.order_type = 'ride'` (or `ride_sharing`). The flow: the passenger requests a fare preview from `calculate-order-pricing` (also re-run authoritatively server-side), then `create-order` inserts the row and fires `start-dispatch` in the background (fire-and-forget). `start-dispatch` runs a round-based radius-escalating search against online riders via the SECURITY DEFINER function `find_nearby_riders`, reading tuning parameters from a cached `dispatch_config` row.

Fare guardrails enforced in `create-order`: a Haversine floor-clamp prevents under-reporting distance below the straight-line distance, and a 5x-straight-line ceiling clamp prevents wildly over-reporting it.

Feature gate: `local_rides_enabled` AND `ride_booking_enabled` (both) — see Part I.

Chapter 18. Local Parcel

`orders.order_type = 'local_parcel'`. Adds parcel-specific columns: `parcel_type`, `receiver_name`/`phone`, `weight_category`, `actual_weight_kg`, `parcel_images[]`, `special_instructions`, `parcel_declared_value`, `high_risk_parcel`, plus compliance columns (`sender_declaration_accepted`, `compliance_flagged`, `compliance_score`). Pickup/dropoff photo evidence is written to `order_parcel_photos` via `process-verification-photo` (or, when a photo genuinely cannot be taken, `bypass-verification-photo` with a mandatory typed reason — see Part VII).

A parcel over 10kg is restricted to auto/cab vehicles — enforced independently in both `calculate-order-pricing` (so the client never even sees a fare for an invalid combination) and `create-order`/`start-dispatch` (defense in depth).

Feature gate: `parcel_enabled` (local parcel only — intercity parcel is governed by `intercity_enabled` instead).

Chapter 19. Intercity Ride & Parcel Matching

Intercity does not use ad-hoc dispatch. A Drop Partner (as "traveller") publishes a `rider_travel_plans` row (a corridor: pickup city/coords, destination city/coords, travel date, weight preference). A passenger's intercity order (still an `orders` row, `order_type` intercity_ride/intercity_parcel) is matched against compatible travel plans by `match-intercity-riders`.

`match-intercity-riders` is **discovery-only** — it never auto-creates a negotiation or sends an automated chat message. A real negotiation row in `intercity_requests` is only created when a user explicitly taps "Travel Together" client-side. Matching logic: city-name match OR Haversine radius match (currently a hardcoded 25km, NOT a `production_config`/database-configurable value despite the function's name) OR route-corridor proximity, filtered by a ±1.5 day date window, vehicle type, parcel weight vs. remaining plan capacity, and available seats. Matches are scored 0–100 (route overlap 50pts, rider reliability 25pts, capacity fit 15pts, online status 10pts).

> **WARNING — Matching radii are hardcoded, not configurable**
> `getIntercitySystemSettings()` inside `match-intercity-riders` returns hardcoded values (25km city-radius match, 15km corridor-radius match). Changing these requires an Edge Function code change and redeploy — there is no `production_config` or database row to edit for this.

The `intercity_requests` status machine is distinct from `orders.order_status`: `request_sent → discussion_started → confirmed → paid → in_progress → completed / cancelled`. Cancellation attribution (who cancelled) is written server-side into `orders.cancelled_by` and `intercity_requests.cancelled_by_actor` in the same UPDATE that changes status — never client-supplied — specifically so cancellation wording in the UI can never be spoofed.

Feature gate: `intercity_enabled` covers both intercity_ride and intercity_parcel; there is no per-type split.

Chapter 20. Partner/Rider Online System & Dispatch

A rider's `account_status` (online/offline/busy/blocked) and live `current_lat`/`current_lng` are written together; a historical bug existed where these were two separate writes, which could leave a rider briefly "online" with stale coordinates in a race — the fix requires the flip and the coordinate write to be atomic (see `nearby_partner_count_regression_test.dart` in the Appendix Change Log).

`find_nearby_riders()` — and any function meant to run with elevated privilege from a client-callable path — must be `SECURITY DEFINER` and must read live `dispatch_config` rather than a hardcoded radius/limit. A real production bug had a passenger-facing "X active partners nearby" count silently return 0 because it ran under the caller's own restrictive RLS instead of definer rights.

Round escalation for local dispatch (`start-dispatch`, config from `dispatch_config`, NOT VERIFIED IN CURRENT PROJECT for live numeric values — `docs/13_Business_Logic.md` itself contains two internally-conflicting value sets for the same profile): Round 1 always runs at the configured radius; Round 2 triggers after a configured elapsed time OR immediately if Round 1 found zero riders; Round 3 only if `allow_round_3` is set. Each round calls `find_nearby_riders`, inserts `rider_assignments` + `order_assignment_logs`, then polls until the round's response timeout elapses.

Chapter 21. Active Rides & Order Lifecycle

`update-order-status` is the single, central state-machine transition function governing everything after acceptance. It is deliberately **not** gated by any `production_config` flag, so that an emergency module disable can never strand an order already past `searching`.

Status lifecycle (code-verified against `docs/07_Payments.md` and `update-order-status` behavior):

Order type	Lifecycle
ride / ride_sharing / intercity_ride	searching → accepted → ongoing → payment_pending → completed (cancellable from most states)
local_parcel / intercity_parcel / batch/food/grocery (unused placeholders)	searching → accepted → rider_arrived → picked_up → rider_at_drop → payment_pending → delivered

`ongoing → completed` is **not** a direct transition for ride orders — `ongoing` only ever leads to `payment_pending`, which alone leads to `completed`. A passenger's rating prompt is only reachable from `completed`.

Concurrency: nearly every transition is optimistic-locked (`UPDATE ... WHERE order_status = <expected current>`); zero rows affected means someone else already changed it, and the caller gets a 409 conflict rather than corrupting state. The final completion step runs inside a single atomic RPC,

`complete_order_and_settle`, which replaces what used to be up to 7 sequential round trips — a real forensic incident found one order took 80 seconds under contention before this fix.

Role-scoped guard: only the assigned driver may revert an accepted order back to `searching` (a rider self-cancel, which triggers redispatch); a passenger calling the same transition is blocked, since passenger-cancellation must not trigger redispatch side effects.

> **WARNING — Audit-trail gap, now fixed going forward**
> `order_status_history.changed_by` is required (`NOT NULL`); callers that historically omitted it silently lost the audit row — a 2026-08-07 investigation found 241 of 285 cancelled orders were missing this row. This is now defended with a `changed_by || order.user_id || 'system'` fallback, but if you are writing new code that transitions an order's status, always pass an explicit `changed_by`.

Chapter 22. Ride Completion, Cancellation & Settlement

Money movement on completion branches on `fare_payment_method`:

- **`personal_upi` / `cash`** — the driver already physically has the money. The platform's commission share is tracked as `cod_pending_amount` on the rider; the wallet is **not** re-credited for the ride earning. Only the driver's own "Confirm Payment Received" attestation (server-enforced: assigned driver only, not before `payment_pending`, not twice) marks the order paid.
- **`cashfree`** — the driver's wallet is credited `rider_earning` via `credit_rider_for_completed_order`, a single atomic `column = column + delta` update. Only a server-verified Cashfree webhook or poll can mark it paid; neither party can self-report a Cashfree payment as complete.

The optimistic-lock update guarantees the completion/credit step runs exactly once regardless of client retries — this is what prevents a double-credit on a flaky network retry.

A legacy `payment_mode='upi'` (a raw `upi://pay` deep link) exists in the schema only so already-in-flight legacy orders can finish; no new order can select it (see the Payments chapter for why it was retired).

Chapter 23. Notifications

Every push-triggering event funnels through one path: a Postgres trigger inserts a `notifications` row inside the same transaction as the business event, then asynchronously (via `pg_net`, never a synchronous call from inside the trigger) calls `send_push_notification_fnc` → `deliver_push_notification_http_fnc` → the `send-push-notification` Edge Function → Firebase Cloud Messaging.

Per-user delivery preferences (read from `users`) gate whether a push actually goes out: `pref_order_notifications`, `pref_chat_notifications`, `pref_rating_notifications`, `pref_admin_notifications`, and — specifically for order-status pushes — `order_notification_mode` (`all` / `important_only` [default] / `tracking_silent`, which only ever pushes on `cancelled`).

The `notification_failures` table (added in the recent "Phase 6" work) is populated only on the failure branch of exactly four functions: `deliver_push_notification_http_fnc`, `on_order_status_update`, `send_order_status_notifications_core`, and `send_push_notification_fnc`. It is additive and admin-only-readable, with no retry mechanism by design. Two trigger functions — `on_intercity_request_change` and `on_rider_review_insert` — deliberately were **not** wrapped with this failure-tracking safety net; that remains a separate, not-yet-approved decision (tracked internally as "Item 5").

> **WARNING — Disabling `notifications_enabled` only stops push**
> Setting `notifications_enabled = false` stops new outbound FCM pushes via `send-push-notification`. It does not touch the already-inserted `notifications` row, so the in-app inbox/history is completely unaffected and nothing is lost — only the phone-buzzing part is suppressed.

Tap-routing (mapping a push's `type` payload to a navigation target) lives in `lib/modules/notifications/services/notification_service.dart`. Commit `1ffd8f1` ("fix: correct notification tap routing") is a real, recent fix to this exact logic — always re-check current source before assuming a specific type→screen mapping documented elsewhere is still accurate.

Chapter 24. Ratings and Feedback

The ratings table is `rider_reviews` (there is no separate "reviews" table). One review per order (enforced by a UNIQUE constraint on `order_id`), rating 1–5, plus an optional tag array (`clean_vehicle`, `polite_behavior`, `safe_driving`, `on_time`, `dirty_vehicle`, `rude_behavior`, `unsafe_driving`, `late_arrival`, `other`) and free-text comment.

`riders.rating` is recalculated on every INSERT/UPDATE/DELETE by a Bayesian-average trigger: `GREATEST(ROUND((SUM(rating)+25)/(COUNT(rating)+5), 2), 3.5)` — a brand-new rider effectively starts at 5.0, and no rider's displayed rating can fall below 3.5 regardless of how many low ratings they receive.

Reviews are immutable once submitted: no UPDATE or DELETE policy exists for any role including admin at the RLS layer — moderation, if ever needed, requires a `service_role` operation, not a client-facing one.

Chapter 25. Subscriptions (WayDrop MAX) & Free Trial

WayDrop MAX is the rider subscription tier: while `riders.subscription_active` is true and `subscription_expires_at` is in the future, the platform's standard commission is waived on that rider's orders (`subscription_commission_waived = true` is set on the order at acceptance time, and the effective commission percentage used in the fare-breakdown recompute becomes 0).

Plans live in `subscription_plans` (PK `plan_id`, a short text code), extended by a later migration to support one active plan per vehicle type. Purchase/entitlement history lives in `rider_subscription_history` (there is no separate "rider_subscriptions" table) — `amount` there is a **snapshot** captured at purchase time, so changing a plan's live price never retroactively affects an already-purchased subscription.

A 30-day free trial exists via `activate_rider_free_trial()`, explicitly noted in the vehicle-specific-pricing migration's own comment as completely out of scope of that pricing change — treat free-trial logic as its own concern, not something to touch while adjusting prices.

> **WARNING — A real, fixed historical vulnerability**
> `create-payment` once trusted a client-supplied `amount` verbatim for `payment_for='subscription'`, and expiry was computed from a client-supplied `metadata.days` value snapped to end-of-calendar-day IST — exploitable for a near-free, near-unlimited zero-commission subscription. It is now fixed by an authoritative server-side plan lookup (`subscription_plans` allowlist by `plan_id`) and a millisecond-precision `computeSubscriptionExpiry()` that stacks onto any existing future expiry rather than snapping to a calendar boundary. Never reintroduce a client-supplied amount or duration for a subscription payment.

> **WARNING — Entitlement activation is idempotent on history-row existence, not payment status**
> Subscription entitlement activation (updating `riders.subscription_active`/`subscription_expires_at` and inserting a `rider_subscription_history` row) must be treated as idempotent based on whether a history row already exists for that `payment_id` — never on `payments.payment_status` alone. A failed `riders` UPDATE after `payment_status` already flipped to `paid` was previously silently unrecoverable, since webhook retries and client re-polls would see "already processed" and skip re-attempting the entitlement grant. A manual recovery recipe exists — see Part IV, Troubleshooting: Subscription Not Activating. No automatic self-healing exists yet for new occurrences of this specific gap.

Chapter 26. Vehicle-Specific Pricing

`subscription_plans` supports a `vehicle_type` column (nullable) with a partial unique index guaranteeing at most one ACTIVE plan per vehicle type. Seeded plan codes at generation time: `daily` (deactivated, kept only as a historical FK target), `weekly` (vehicle-agnostic), `daily_bike`, `daily_auto`, `daily_cab`. A subscription plan tied to a specific `vehicle_type` requires the purchasing rider to have that vehicle verified — either via an approved `rider_kyc` row or an approved `rider_vehicles` row — enforced server-side in `create-payment` with no fallback to a different plan; a mismatch fails closed with a 403.

See Part IV, Changing Subscription Prices, for the exact recipe.

Chapter 27. Payments & Cashfree

WayDrop's real, primary settlement path for ride/parcel fares is **driver-collected**, not the payment gateway: a driver's own uploaded Personal QR image (shown fullscreen for the passenger to scan and pay manually) or cash. Cashfree is the fallback/optional path, used for: (1) subscription purchases (always gateway-based), and (2) passengers who prefer to pay the fare through the gateway, either via full hosted checkout or `create-fare-qr`'s in-checkout "Scan and Pay" QR.

> **WARNING — Retired — do not reintroduce**
> An earlier architecture generated a raw `upi://pay` deep link with an in-app-rendered QR from a typed VPA. It was killed because GPay/PhonePe/NPCI flagged a personal VPA receiving many small payments from rotating strangers as mule-account-pattern activity, causing real on-device blocks. `payment_mode='upi'` and `upi_payment_prompted_at` remain in the schema only so already-in-flight legacy orders can complete — no new order can select this path, and no future code should fabricate or reconstruct a raw UPI payload client-side.

Money-movement authority is strict: for Personal QR/Cash, **only** the assigned driver's own "Confirm Payment Received" tap can mark an order paid; for Cashfree, **only** a server-verified webhook or poll can — neither party can ever self-report a Cashfree payment as successful.

The webhook (`cashfree-webhook`) verifies a timing-safe HMAC-SHA256 signature computed over the raw, unparsed request body — re-serializing the JSON before verifying would break the signature, so the raw body is always read as text first. `verify-payment` (client-triggered polling while waiting for the webhook) is functionally a mirror of the webhook's success-handling logic, deliberately kept identical since either path can be the one that first observes a successful payment.

Duplicate-session protection: `create-payment`/`create-fare-qr` check for an existing pending/processing payment for the same `(user, payment_for, reference_id)` and live-check Cashfree's own order status before deciding whether to reuse it or mint a new one — this specifically fixed a bug where a stale local "pending" row was handed back forever even after the real Cashfree session had expired.

> **WARNING — Never leak a raw Cashfree error to a driver or passenger**
> Every Cashfree failure is logged in full server-side (e.g. `payments.metadata.cashfree_qr_last_error`) and surfaced to the end user only as a fixed, non-technical message. This is a deliberate, tested rule — do not "helpfully" surface a raw gateway error code or message in any future UI change.

`create-fare-qr` does not generate its own QR image — Cashfree's dedicated raw-QR API is permanently blocked on WayDrop's merchant account (`s2s_enabled_not_approved`, requires a separate PCI-DSS approval WayDrop does not have). Instead it creates a UPI-restricted Cashfree hosted-checkout session, and Cashfree's own checkout UI renders the genuine per-order "Scan and Pay" QR inside it.

Chapter 28. Government Tax

A WayDrop-configured (not independently legally verified — explicitly pending CA/accounting confirmation per its own migration header) per-order tax is tracked in `government_tax_ledger`, an append-only, auditable table. `entry_type` is one of `ORDER_TAX`, `ORDER_TAX_WITHHELD`, `TAX_PAYMENT`, or `ADJUSTMENT`; unique partial indexes guarantee exactly one `ORDER_TAX` per order and at most one `TAX_PAYMENT` per `payment_id`, which is the database-level defense against a duplicate webhook/completion replay double-charging or double-crediting tax.

`riders.government_tax_due_balance` is a cached running total of this ledger, always updated atomically (`column = column + delta`) in the same transaction as the ledger insert — it is never written independently. `is_rider_blocked_by_tax_due()` blocks a rider from accepting new orders once their balance reaches (not merely exceeds) the configured threshold. Rates are versioned by `effective_from` in `tax_rate_config`, so a rate change never recalculates historical ledger entries. `intercity_ride` is deliberately left unclassified/unseeded by design — it must never silently default to either the local-ride or parcel rate.

All mutation goes exclusively through SECURITY DEFINER functions (`apply_order_government_tax`, `apply_government_tax_payment`) — never a direct UPDATE.

Chapter 29. Payouts & Withdrawals

A rider first registers a payout destination via `create-rider-payout` (bank account or UPI VPA), which mints a Cashfree Payouts "beneficiary." A UPI VPA is accepted purely on the rider's own self-certification checkbox — WayDrop's Cashfree merchant account does not have the licensed "Verification Suite" to actually verify UPI ownership, so no "Verified" badge is ever shown for that path.

A withdrawal itself (`create-rider-withdrawal`) is protected end-to-end by three SECURITY DEFINER functions with row-level locking on `riders` to serialize concurrent wallet writes:

- `initiate_withdrawal(amount, user_id)` — atomically checks the wallet balance and reserves the amount **before** any external Cashfree call is made.
- `fail_withdrawal(withdrawal_id, error_response)` — refunds the reserved amount back to the wallet on any failure path (missing secret, signature error, network failure, Cashfree rejection). A transfer not resolved by Cashfree within 5 minutes is failed locally specifically so funds are never left permanently locked.
- `complete_withdrawal(withdrawal_id, cashfree_ref_id, status, response)` — sets the final status; refunds the wallet if the final status resolves to failed; both this and `fail_withdrawal` are no-ops if the withdrawal is already finalized, so a retried callback can never double-refund.

> **WARNING — A real, fixed historical bug**
> Cashfree's `conflict_with_existing_beneficiary` error means the bank details (not the beneficiary ID) already exist under a different beneficiary ID. This was previously misclassified as a generic "already exists, treat as success," writing a fabricated "active" status for a beneficiary Cashfree never actually created — every subsequent withdrawal attempt then failed with "beneficiary_not_found." The fix looks up the real beneficiary ID Cashfree already has on file (by account number + IFSC) and reuses it. If you ever see this specific pattern again, it points at the same class of bug.

Every payout-account field change is rate-limited to at most 2 changes in a rolling 7-day window (`check_payout_update_limit`), a deliberate fraud/error mitigation — do not bypass this via direct SQL without understanding why it exists.

Chapter 30. KYC and Documents

`rider_kyc` holds Aadhaar/PAN/driving-license numbers and vehicle registration details, plus 10 uploaded document images, `kyc_status` (pending/under_review/approved/rejected/resubmitted), and `activation_status` (inactive/profile_pending/active/suspended). It is the single highest-sensitivity table in the schema.

Submission (`create-rider-kyc`) enforces: exact-format Aadhaar (12 digits) and PAN (regex-checked) numbers, an age gate (18+, computed from date of birth), and a resubmission cooldown — after 2 rejections, resubmission is blocked for 7 days (`blocked_until`). A pre-flight check (`validate-rider-kyc`) lets the client verify a submission would be accepted without writing anything.

There is **no in-app admin KYC review screen** in the current Flutter app (confirmed by repository-wide grep). KYC approval today is a Supabase Dashboard-only manual operation: Table Editor on `rider_kyc` plus the Storage browser on the `rider_kyc` bucket. See Part IV for the exact recipe.

Chapter 31. Partner Vehicles

`rider_vehicles` lets a rider register additional vehicles beyond the one originally submitted with their KYC (e.g. a bike plus a car). The original `rider_kyc` vehicle is still treated as the rider's first, already-approved vehicle and is read directly — there is no backfill of it into `rider_vehicles`. Approval of a new vehicle (`status`: pending/approved/rejected) has no client-facing UPDATE policy, implying an admin/service-role-only approval path, mirroring KYC.

Chapter 32. User/Partner Profiles & Account Deletion

Account deletion (`delete-account`) is server-authoritative: identity is derived strictly from the caller's own JWT, and the actual anonymization runs through a SECURITY DEFINER RPC (`delete_own_account()`) using `auth.uid()` internally — even this Edge Function cannot act on anyone else's account.

A legally significant design point: KYC identity documents (Aadhaar/PAN/DL/RC/rider photo/vehicle photo) are **archived, not deleted** — copied to a private, admin/support-only retention bucket and then removed from the public-facing path, specifically so a Drop Partner cannot destroy identity evidence a later fraud/safety/legal investigation might need simply by deleting their account. A passenger's profile photo and a driver's personal UPI QR, which are not KYC evidence, are plain-deleted.

The underlying `auth.users` row is never hard-deleted (doing so would violate a foreign-key constraint from any account with order history) — it is instead permanently banned (~100 years) with all sessions revoked, and the phone number is reassigned to a deterministic placeholder so it can be reused for a fresh registration. Deletion is blocked (409, not 500) while the account has an active order in progress.

PART IV

ADMIN OPERATIONS

Chapter 33. Feature Kill-Switches

All emergency controls live in the single `production_config` table (Part II has the full column reference). This chapter gives the exact operational recipe for each one. Every recipe below is the same shape: read the current row, update exactly the one column you mean to change, and confirm.

READ-ONLY

```
-- Always start here: see the live config row before changing anything.

SELECT * FROM public.production_config WHERE is_active = true;
```

Maintenance Mode

PRODUCTION

```
UPDATE public.production_config

SET maintenance_mode = true,

    maintenance_message = 'WayDrop is undergoing scheduled maintenance. Back shortly.'

WHERE is_active = true;
```

Effect: `AppStartScreen.checkUserFlow()` checks this before even checking login state, so both logged-in and logged-out users are blocked behind `MaintenanceScreen` on their next cold start. It does not affect a session already open inside the app, and it does not touch any order's server-side lifecycle — an accepted ride in progress keeps working normally for anyone who doesn't restart the app. Re-enable by setting it back to `false`.

WARNING — This does not evacuate active sessions

Because there is no live-push mechanism to already-open clients, a rider or passenger already mid-session will not be interrupted. Maintenance mode is a "stop new sessions from starting" switch, not a "freeze everything right now" switch.

Force Update / Minimum & Latest Android Version

PRODUCTION

```
-- Hard-block everyone below 1.4.0 (compares the app's version string, not build number):

UPDATE public.production_config

SET minimum_android_version = '1.4.0',

    update_message = 'Please update WayDrop to continue.'

WHERE is_active = true;



-- OR force everyone regardless of version:

UPDATE public.production_config SET force_update = true WHERE is_active = true;



-- Non-blocking "update available" nudge only (no lockout):

UPDATE public.production_config SET latest_android_version = '1.5.0' WHERE is_active =
    true;
```

`force_update = true` OR `installed version < minimum_android_version` routes to `UpdateRequiredScreen`, which has no retry/continue button and no Play Store link by design — the only valid exit is actually updating the app. `latest_android_version` alone (without force_update or raising minimum) only shows a one-time-per-session non-blocking snackbar suggesting an update.

Disable / Enable Local Rides, Parcel, Intercity

PRODUCTION

```
UPDATE public.production_config SET local_rides_enabled = false WHERE is_active = true;

UPDATE public.production_config SET ride_booking_enabled = false WHERE is_active = true;

UPDATE public.production_config SET parcel_enabled = false WHERE is_active = true;

UPDATE public.production_config SET intercity_enabled = false WHERE is_active = true;
```

Each flip greys out the corresponding tile on the passenger home screen (`home_tab.dart`) on next app view, AND makes `create-order`/`accept-order`/`accept-intercity-request` return a 503 `FEATURE_DISABLED` for new orders of that type server-side — so even an old cached app build, or a request replayed outside the app, is still blocked. Re-enable by setting back to `true`.

SAFE

None of these ever touch `update-order-status` — an order already `accepted` (or later) for a disabled module keeps running its full lifecycle to completion untouched. This is intentional and tested (`phase2_production_safety_regression_test.dart`).

Disable / Enable Payments & Subscriptions

PRODUCTION

```
UPDATE public.production_config SET payments_enabled = false WHERE is_active = true;

UPDATE public.production_config SET subscriptions_enabled = false WHERE is_active = true;
```

`payments_enabled = false` blocks ALL new Cashfree payment-session creation via `create-payment` (order fares, subscriptions, tax-due repayment, COD settlement) — there is deliberately no client-side UI toggle for this; the 503 error message returned by the Edge Function IS the intended user-facing message. `subscriptions_enabled = false` is an additional, narrower gate required only for `payment_for='subscription'` specifically.

WARNING — Do not let a UI error handler swallow this message

A previously-fixed bug had `PaymentService.createPayment()`'s error handler discard the server's specific rejection text for exactly this case, showing a generic "Unable to create payment session" instead of the intended message. If you are touching payment error handling, preserve the server's message for a 503.

Disable / Enable Notifications

PRODUCTION

```
UPDATE public.production_config SET notifications_enabled = false WHERE is_active = true;
```

Stops new outbound FCM pushes via `send-push-notification` only. The `notifications` row is still inserted and the in-app inbox is completely unaffected — nothing is lost, phones simply stop buzzing until re-enabled.

Chapter 34. Admin Read-Only Inspection Recipes

All of the following are read-only SELECT queries, safe to run against production directly (Supabase SQL editor or any read-only client). None of them require any deploy.

Viewing active partners (riders)

READ-ONLY

```
SELECT id, user_id, account_status, current_lat, current_lng, last_location_updated_at,

       rating, subscription_active, subscription_expires_at

FROM public.riders

WHERE account_status = 'online'

ORDER BY last_location_updated_at DESC;
```

Viewing active rides

READ-ONLY

```
SELECT id, order_code, order_type, order_status, user_id, rider_id, created_at,
    accepted_at

FROM public.orders

WHERE order_status NOT IN ('completed','delivered','cancelled','expired')

ORDER BY created_at DESC;
```

Viewing orders stuck in searching

READ-ONLY

```
SELECT id, order_code, order_type, created_at, search_expires_at, dispatch_attempts,

       current_search_radius_km, dispatch_worker_running

FROM public.orders

WHERE order_status = 'searching'

ORDER BY created_at ASC;
```

A `dispatch_worker_running = true` with `dispatch_worker_started_at` more than a few minutes old suggests a stuck/crashed dispatch loop — see Part V, Ride Stuck in Searching.

Viewing intercity requests

READ-ONLY

```
SELECT id, status, initiated_by, proposed_fare, sender_confirmed, traveller_confirmed,
    parcel_id

FROM public.intercity_requests

ORDER BY id DESC

LIMIT 100;
```

Viewing KYC documents and photos

See the full KYC Cheat Sheet in Part X. In short: `SELECT * FROM public.rider_kyc WHERE id = '<rider_kyc_id>';` to see status and stored image paths, then open the Storage browser on the `rider_kyc` bucket at that same path to view the actual images — there is no in-app admin viewer.

Viewing withdrawals and payout history

READ-ONLY

```
SELECT w.id, w.transfer_id, w.status, w.amount, w.created_at, w.updated_at,
    w.cashfree_reference_id

FROM public.rider_withdrawals w

WHERE w.rider_id = '<rider_id>'

ORDER BY w.created_at DESC;



SELECT * FROM public.rider_withdrawal_events WHERE withdrawal_id = '<withdrawal_id>' ORDER
    BY created_at;
```

Viewing payments

READ-ONLY

```
SELECT id, payment_code, payment_for, payment_status, amount, final_amount,

       gateway_order_id, gateway_transaction_id, paid_at, failure_reason

FROM public.payments

WHERE user_id = '<user_id>' OR rider_id = '<rider_id>'

ORDER BY created_at DESC;
```

Viewing subscriptions

READ-ONLY

```
SELECT h.*, p.plan_id, p.label, p.amount

FROM public.rider_subscription_history h

JOIN public.payments pay ON pay.id = h.payment_id

LEFT JOIN public.subscription_plans p ON p.plan_id = pay.plan_id

WHERE h.rider_id = '<rider_id>'

ORDER BY h.starts_at DESC;
```

Viewing ratings

READ-ONLY

```
SELECT * FROM public.rider_reviews WHERE rider_id = '<rider_id>' ORDER BY created_at DESC;
```

Viewing notifications and failed notifications

READ-ONLY

```
SELECT * FROM public.notifications WHERE user_id = '<user_id>' ORDER BY created_at DESC
    LIMIT 50;



-- Failures are admin-only readable and additive-only:

SELECT * FROM public.notification_failures ORDER BY created_at DESC LIMIT 100;
```

Viewing payment failures

READ-ONLY

```
SELECT * FROM public.payments WHERE payment_status = 'failed' ORDER BY created_at DESC
    LIMIT 100;
```

Chapter 35. Investigating Stuck Orders

1. Find the order and its current status (query above).
2. If `searching`: check `dispatch_worker_running`/`dispatch_worker_started_at` — a lock older than 120 seconds is treated as stale and self-heals on the next dispatch trigger; if it's been stuck much longer, check whether `local_rides_enabled`/`parcel_enabled`/`intercity_enabled` was accidentally left disabled (new-order gate, not relevant to an existing order, but worth ruling out if riders report no offers at all).
3. If `accepted`/`ongoing`/etc. and genuinely frozen (not just slow): check Realtime connectivity for that specific client first (Part V) before assuming server-side breakage — `update-order-status` is optimistic-locked, so a client retry against a stale cached status will get a clean 409, not silent corruption.
4. If `payment_pending` and stuck: confirm whether the intended settlement is Cashfree (check for a matching `payments` row and its `payment_status`) or driver-attested (personal_upi/cash — confirm the driver actually tapped "Confirm Payment Received"). Never manually flip `orders.order_status` to `completed` — always go through the same transition path a client would use, or, if diagnosing only, leave the order in place and check the appropriate Part V chapter.
5. Check `order_status_history` for that order to see the exact sequence of transitions and who triggered each one.

DANGEROUS - CONFIRM BEFORE RUNNING

Never directly `UPDATE orders SET order_status = ...` to "unstick" an order. Several columns on `orders` are trigger-protected, and completion specifically must go through `complete_order_and_settle` to correctly settle wallet/tax/commission side effects — a raw UPDATE can silently desynchronize the rider's wallet, the government tax ledger, and the audit history from the order's new state.

Chapter 36. Changing Subscription Prices

SAFE — This is the documented, supported way to change a price

```
-- 1. Confirm current plans and prices first:

SELECT plan_id, label, amount, duration_hours, vehicle_type, is_active

FROM public.subscription_plans

ORDER BY plan_id;


-- 2. Change exactly one plan's price:

UPDATE public.subscription_plans

SET amount = 49

WHERE plan_id = 'daily_bike';
```

Question	Answer
Exact table	`public.subscription_plans`
Exact plan IDs (as seeded by migrations — confirm live via the SELECT above)	`weekly` (vehicle-agnostic), `daily_bike`, `daily_auto`, `daily_cab`, plus `daily` (deactivated, kept only as a historical FK target — do not reactivate it without understanding why it was deactivated)
Which column controls the amount	`amount` (numeric)
Does `is_active` matter	Yes — only ACTIVE plans are shown to purchasers (RLS restricts client SELECT to `is_active = true`), and a partial unique index enforces at most one active plan per `vehicle_type`. Deactivate a plan by setting `is_active = false`, never by deleting it.
Are existing subscribers affected	No. `rider_subscription_history.amount` is a snapshot taken at purchase time — a price change only ever affects future purchases.
Are new purchases only affected	Yes, immediately — `create-payment` looks up the current `amount` by `plan_id` at the moment of purchase, with no caching layer to invalidate.
Edge Function deploy required	No.
Migration required	No — this is a plain data UPDATE, not a schema change.
APK rebuild required	No — the price is never hardcoded client-side; the client displays whatever `subscription_plans` currently returns.

DANGEROUS - CONFIRM BEFORE RUNNING

Do not change `plan_id` (it is the foreign-key target for every historical `payments`/`rider_subscription_history` row referencing that plan). Do not delete a plan row — deactivate it. Do not hand-edit `duration_hours` for a plan that already has active subscribers without first understanding the expiry-recalculation implications for those in-flight subscriptions.

PART V

TROUBLESHOOTING

Chapter 37. Local Ride Booking Not Working

PROBLEM: Passengers cannot start/finish a local ride booking.

FIRST CHECK: `production_config.local_rides_enabled` and `ride_booking_enabled` — both must be true. If either is false, `create-order` returns 503 `FEATURE_DISABLED` and the home tile is greyed out client-side.

TABLES: `orders`, `production_config`, `pricing_zones`, `pricing_config`.

FUNCTIONS: `create-order`, `calculate-order-pricing`, `start-dispatch`.

READ-ONLY

```
SELECT local_rides_enabled, ride_booking_enabled FROM public.production_config WHERE
    is_active = true;
```

APP FILES: `lib/features/local_rides/booking/screens/ride_location_screen.dart`.

EXPECTED RESULT: Both flags true, `create-order` returns `{success:true, order}`.

IF THIS IS WRONG: If a fare of 0 or a pricing error appears instead, this is a pricing/zone problem, not a booking-disabled problem — check `calculate-order-pricing`'s zone resolution (a pickup point outside any `pricing_zones` geofence and not matching a zone's city list will fail unless the caller is in Developer Test Mode).

SAFE FIX: Re-enable the flag via the recipe in Part IV.

DO NOT TOUCH: `orders.order_status` directly.

WHEN APK IS REQUIRED: No.

WHEN EDGE FUNCTION DEPLOY IS REQUIRED: No, unless the root cause is a bug in `create-order`/`calculate-order-pricing` itself.

WHEN MIGRATION IS REQUIRED: No, unless a `pricing_zones`/`pricing_config` row is genuinely missing for the area (that's a data INSERT, not a migration).

Chapter 38. Local Parcel Not Working

Same triage as local ride booking, substituting `parcel_enabled` for the flag check and `lib/features/local_parcels/booking/screens/parcel_booking_screen.dart` for the app file. Additionally check: a parcel over 10kg requested on a bike vehicle type is rejected by design (both client-side pricing and server-side `create-order`) — this is expected behavior, not a bug.

Chapter 39. Intercity Matching Not Working

PROBLEM: A traveller's posted travel plan or a passenger's intercity request isn't finding matches.

FIRST CHECK: `production_config.intercity_enabled`. Then confirm the traveller's `riders.service_mode` is actually `'intercity'` — matching explicitly filters on this.

TABLES: `rider_travel_plans`, `intercity_requests`, `travel_matches`, `orders`.

FUNCTIONS: `match-intercity-riders`.

READ-ONLY

```
SELECT id, pickup_city, destination_city, travel_date, status

FROM public.rider_travel_plans

WHERE rider_id = '<rider_id>' AND status = 'active';
```

EXPECTED RESULT: An active plan within about 1.5 days of the requested date, matching vehicle type, with remaining capacity, on the same or a nearby corridor (25km city-radius or 15km corridor-radius — both hardcoded in `match-intercity-riders`, not configurable without a redeploy).

IF THIS IS WRONG: Remember matching is discovery-only — it will surface a "Traveller Available" notification, but a real `intercity_requests` row only appears after someone explicitly taps "Travel Together." If a passenger says "nothing happened after I tapped a match," that click, not the matching engine, is the next thing to check (`intercity_matching_service.dart`, `sendTravelTogetherRequest`).

SAFE FIX: None beyond re-enabling `intercity_enabled` if disabled — matching radii require a code change.

DO NOT TOUCH: Do not hand-insert an `intercity_requests` row to "force" a match — this bypasses the negotiation flow both parties expect to see.

WHEN EDGE FUNCTION DEPLOY IS REQUIRED: Yes, if the actual fix is changing the matching radius or scoring weights (hardcoded in source).

Chapter 40. Partner Cannot Go Online / Cannot Accept Ride

PROBLEM: A Drop Partner can't flip online, or taps accept and nothing happens.

FIRST CHECK: `riders.account_status`, `riders.kyc_id` → `rider_kyc.kyc_status`/`activation_status`. A rider without an approved, active KYC row cannot go online.

TABLES: `riders`, `rider_kyc`, `rider_assignments`.

FUNCTIONS: `accept-order`, `find_nearby_riders`.

READ-ONLY

```
SELECT r.account_status, r.subscription_active, k.kyc_status, k.activation_status,
    k.blocked_until

FROM public.riders r JOIN public.rider_kyc k ON k.id = r.kyc_id

WHERE r.id = '<rider_id>';
```

EXPECTED RESULT: `kyc_status = 'approved'`, `activation_status = 'active'`, `blocked_until` null or in the past.

IF THIS IS WRONG: An "accept" that silently fails with `WRONG_SERVICE_MODE`, `TAX_DUE_THRESHOLD_BLOCKED`, or `COD_LIMIT_EXCEEDED` is a structured, intentional rejection from `accept-order` — read the exact `code` in the response before assuming a bug. Check `riders.government_tax_due_balance` against the active threshold, and `riders.cod_pending_amount` against `cod_limit` (default 500), respectively.

SAFE FIX: If genuinely a KYC data issue, correct it via the KYC Cheat Sheet in Part X. If a tax-due or COD-limit block, the rider needs to actually pay down the balance (Part III, Government Tax) — do not manually zero the balance to unblock them.

DO NOT TOUCH: `riders.wallet_balance`, `government_tax_due_balance`, `cod_pending_amount` directly.

Chapter 41. Ride Stuck in Searching / Stuck After Acceptance

See **Investigating Stuck Orders** in Part IV for the full recipe. In short: `searching` stuck usually means no eligible online riders were found within the configured radius/rounds, or `dispatch_worker_running` is stuck true past its 120-second staleness window (self-heals on the next trigger). Stuck **after** acceptance is not a dispatch problem at all — check Realtime connectivity for the specific client and `order_status_history` for the real sequence of events.

Chapter 42. Payment Not Working / Cashfree Webhook Problems

PROBLEM: Cashfree confirms a payment but the order doesn't reflect it, or a payment session can't be created at all.

FIRST CHECK: `production_config.payments_enabled` (and `subscriptions_enabled` for a subscription specifically) for session-creation failures. For a webhook mismatch, check the Edge Function logs for `cashfree-webhook` for a signature verification failure.

TABLES: `payments`, `orders`.

FUNCTIONS: `create-payment`, `cashfree-webhook`, `verify-payment`.

READ-ONLY

```
SELECT id, payment_code, payment_status, gateway_order_id, webhook_received,
    failure_reason

FROM public.payments WHERE payment_code = '<payment_code>';
```

EXPECTED RESULT: `payment_status = 'paid'`, `webhook_received = true` once Cashfree confirms.

IF THIS IS WRONG: A webhook signature mismatch (400 from `cashfree-webhook`) is almost always a `CASHFREE_SECRET_KEY` mismatch between what's configured in Supabase secrets and what Cashfree is actually using — verify via `supabase secrets set CASHFREE_SECRET_KEY=...` (this auto-restarts the function runtime). Remember the webhook signature is computed over the RAW unparsed body — a proxy or logging layer that re-serializes JSON before it reaches the handler will break every signature. `verify-payment` (client poll) uses the same success-handling logic as the webhook and can independently resolve a payment if the webhook is delayed or lost — a stuck payment does not always mean the webhook is broken.

SAFE FIX: Have the client re-trigger `verify-payment`. If the payment is confirmed paid on the Cashfree dashboard but never reflected in `payments`, this is the documented "Phase 5 Gap D" — see the Subscriptions troubleshooting entry below for the recovery recipe (the same idempotency mechanism applies to order-fare payments).

DO NOT TOUCH: Never manually set `payments.payment_status = 'paid'` for a fare payment without independently confirming with Cashfree that money actually moved — this is exactly the class of manual

"fix" `docs/16_Runbook.md`'s stale webhook-reconciliation recipe warns needs re-verification against the current schema before reuse.

Chapter 43. Subscription Not Activating / Price Wrong / Free Trial Issues

PROBLEM: A rider paid for WayDrop MAX but `subscription_active` never flipped true.

FIRST CHECK: Does a `rider_subscription_history` row exist for that `payment_id`? If the payment is `paid` but no history row exists, this is the known "Phase 5 Gap D" reliability gap — entitlement activation is not yet automatically retried.

TABLES: `payments`, `subscription_plans`, `rider_subscription_history`, `riders`.

READ-ONLY

```
SELECT p.id, p.payment_status, p.plan_id, h.id AS history_id

FROM public.payments p

LEFT JOIN public.rider_subscription_history h ON h.payment_id = p.id

WHERE p.rider_id = '<rider_id>' AND p.payment_for = 'subscription'

ORDER BY p.created_at DESC LIMIT 5;
```

SAFE FIX — manual recovery (code-verified recipe):

PRODUCTION

```
-- 1. Look up duration_hours from subscription_plans by the payment's plan_id (never guess
    it).

-- 2. Extend/activate the subscription, stacking onto any existing future expiry:

UPDATE public.riders

SET commission_waiver_active = true,

    subscription_active = true,

    subscription_expires_at = GREATEST(NOW(), COALESCE(subscription_expires_at, NOW()))

                              + (<duration_hours> || ' hours')::interval

WHERE id = '<rider_id>';



-- 3. Insert the history row so this payment can never be double-applied:

INSERT INTO public.rider_subscription_history (rider_id, payment_id, amount, starts_at,
    expires_at, is_active)

VALUES ('<rider_id>', '<payment_id>', <amount>, NOW(), <new_expires_at>, true)

ON CONFLICT DO NOTHING;
```

WARNING

The `ON CONFLICT DO NOTHING` above relies on a real unique index on `payment_id` (`idx_unique_subscription_payment`) — this recipe is only safe because that index exists, making a re-run of this exact recovery for the same payment a safe no-op rather than a double-grant.

For "price wrong" — confirm the live `subscription_plans.amount` first (Part IV recipe); remember a price change never retroactively affects an already-purchased `rider_subscription_history` snapshot, so a rider quoting an old price they already paid is not a bug. For free-trial issues, `activate_rider_free_trial()` is explicitly a separate concern from paid-plan pricing — don't conflate the two when diagnosing.

Chapter 44. KYC Documents / Photos Not Showing

PROBLEM: A rider's photo (profile or vehicle) doesn't display in the app, or an admin can't view a submitted document.

FIRST CHECK: Only `rider_image.jpg` and `vehicle_image.jpg` are ever rendered client-side — the other 8 KYC images (Aadhaar/PAN/DL/RC) are never shown in the app by design; "not showing" for those is expected, not a bug. For the two that should show: the `rider_kyc` bucket has been private since 2026-07-31, so every legitimate display path must resolve a fresh signed URL — a hardcoded old public URL will now 403.

TABLES: `rider_kyc`.

APP FILES: `kyc_image_resolver.dart` (signed URL resolution).

SAFE FIX: Confirm the stored value in `rider_kyc` is a bare storage path (current format) rather than a stale full public URL (legacy format) — both are handled by the resolver, but if a row somehow has neither, the underlying file may be missing from Storage.

DO NOT TOUCH: Do not flip the `rider_kyc` bucket back to public to work around this — that reintroduces the exact P1-2 security issue that was fixed.

Chapter 45. Notification Not Arriving / Tap Does Nothing

PROBLEM: A push never shows up, or shows up but tapping it does nothing / goes to the wrong screen.

FIRST CHECK: `production_config.notifications_enabled`. Then the recipient's own preference columns on `users` (`pref_order_notifications` etc., and `order_notification_mode` for order-status pushes specifically — `important_only` intentionally suppresses many status pushes).

TABLES: `notifications`, `notification_failures`, `user_device_tokens`.

READ-ONLY

```
SELECT delivery_status, push_sent, delivery_error FROM public.notifications

WHERE user_id = '<user_id>' ORDER BY created_at DESC LIMIT 10;



SELECT * FROM public.notification_failures WHERE user_id = '<user_id>' ORDER BY created_at
    DESC LIMIT 10;
```

IF THIS IS WRONG: `delivery_status = 'pending'` for a long time with no matching `notification_failures` row means the failure happened somewhere `notification_failures` doesn't cover — specifically `on_intercity_request_change` or `on_rider_review_insert`, the two trigger paths deliberately not wrapped by the Phase 6 safety net. `push_sent = false` with a real error usually means the FCM token in

`user_device_tokens` is stale (self-cleans on the next attempt if FCM reports it unregistered).

For "tap does nothing / wrong screen": this is a client-side routing issue in `lib/modules/notifications/services/notification_service.dart`, not a data problem — commit `1ffd8f1` recently fixed exactly this class of bug, so re-check current source before assuming the mapping documented elsewhere is still accurate.

WHEN APK IS REQUIRED: Yes, for a tap-routing fix (it's client-side Dart code).

Chapter 46. Rating Feedback Not Showing

PROBLEM: A submitted rating/review isn't visible somewhere it's expected.

FIRST CHECK: `rider_reviews` RLS only allows the reviewing user or the rated rider to SELECT a given review — there is no broader admin/public read policy in the base migration. "Not showing to a third party" may be RLS working as intended, not a bug.

TABLES: `rider_reviews`, `riders` (for the recalculated `rating` column).

READ-ONLY

```
SELECT * FROM public.rider_reviews WHERE order_id = '<order_id>';

SELECT rating FROM public.riders WHERE id = '<rider_id>';
```

DO NOT TOUCH: Never UPDATE or DELETE a `rider_reviews` row directly — no policy permits it for any client-facing role, by design; reviews are immutable once submitted.

Chapter 47. Withdrawal Not Working

PROBLEM: A rider's withdrawal is stuck pending, or failed unexpectedly.

FIRST CHECK: `rider_withdrawals.status` and the full `rider_withdrawal_events` trail for that withdrawal.

TABLES: `rider_withdrawals`, `rider_withdrawal_events`, `rider_payout_accounts`.

READ-ONLY

```
SELECT * FROM public.rider_withdrawals WHERE id = '<withdrawal_id>';

SELECT * FROM public.rider_withdrawal_events WHERE withdrawal_id = '<withdrawal_id>' ORDER
    BY created_at;
```

EXPECTED RESULT: `status` progresses pending → processing → success, with wallet balance already deducted at `pending` (reserved atomically by `initiate_withdrawal`).

IF THIS IS WRONG: `status = 'failed'` should always correspond to a wallet refund already applied by `fail_withdrawal` — confirm `riders.wallet_balance` reflects that. A transfer genuinely unresolved on Cashfree for more than 5 minutes is automatically failed-and-refunded locally rather than left open indefinitely. `conflict_with_existing_beneficiary` on the underlying payout-account setup is a known historical failure class — see Part III, Payouts & Withdrawals, for the exact fix already applied in `create-rider-payout`.

DO NOT TOUCH: Never manually UPDATE `rider_withdrawals.status` or `riders.wallet_balance` — always resolve via `complete_withdrawal`/`fail_withdrawal` so the wallet and the audit trail stay consistent.

Chapter 48. Cashfree Webhook Problem

See "Payment Not Working" above for the full recipe. The single most common root cause is a `CASHFREE_SECRET_KEY` mismatch between Supabase Edge Function secrets and Cashfree's dashboard — always verify the secret value first via `supabase secrets set` before assuming a code bug in the signature-verification logic itself.

Chapter 49. Government Tax Problem

PROBLEM: A rider disputes their tax-due balance, or was unexpectedly blocked from accepting orders.

FIRST CHECK: `riders.government_tax_due_balance` against the active `government_tax_config.threshold_amount` — blocking triggers at greater-than-or-equal, not strictly greater-than.

READ-ONLY

```
SELECT government_tax_due_balance FROM public.riders WHERE id = '<rider_id>';

SELECT threshold_amount FROM public.government_tax_config WHERE is_active = true;

SELECT * FROM public.government_tax_ledger WHERE rider_id = '<rider_id>' ORDER BY
    created_at DESC;
```

DO NOT TOUCH: Never manually INSERT/UPDATE `government_tax_ledger` or `riders.government_tax_due_balance` directly — always use `apply_order_government_tax`/`apply_government_tax_payment`, which maintain the unique-per-order/per-payment guarantees that prevent double-charging.

Chapter 50. Realtime Problem

PROBLEM: Live order/rider-position updates stop arriving; the app looks frozen on a tracking screen.

FIRST CHECK: The client's Realtime connection state (connected/disconnected/reconnecting) and whether the affected table (`orders`, `riders`, `order_chat_messages`) still has `REPLICA IDENTITY FULL` and is part of the `supabase_realtime` publication.

APP FILES: `lib/core/services/realtime/` (recently hardened per commit `e1c0f8e`).

SAFE FIX: A client-side reconnect (backgrounding/foregrounding the app) usually recovers a dropped WebSocket. If it does not, treat this as a server-side Realtime/publication configuration issue, not a data issue.

Chapter 51. App Appears Slow

Several historical performance investigations exist in this codebase's commit history (see the Appendix Change Log — commits `97145e9`, `1c889a4`, `41f3bc6`, `24b3bd2`). Before assuming a new performance regression, check whether the specific hot path you're investigating (rider active-order polling, notification delivery, dispatch round timing) already has a documented fix. `start-dispatch`'s own instrumentation logs are explicitly tied to a past "order created → dispatch begins" latency investigation — its console timing traces are there for exactly this kind of debugging.

Chapter 52. Duplicate Requests / Duplicate Notifications

Most write paths in this codebase are deliberately idempotent against exactly this: `create-order` (idempotency_key), `create-payment`/`create-fare-qr` (existing pending-session reuse), `match-intercity-riders` (unique `(order_id, plan_id)` match ledger), withdrawal RPCs (finalized-state no-ops). A duplicate you're seeing is more likely a genuinely new event (e.g. a second real notification-worthy trigger firing) than a broken idempotency guard — check `notifications.reference_id`/`reference_type` to confirm whether two rows are truly duplicates of the same event or two distinct events before treating it as a bug.

Chapter 53. Production Emergency Response

1. Identify scope (one user vs. everyone) using the Part I triage funnel.
2. If a whole module is affected and you need to buy time immediately, use the appropriate `production_config` kill-switch from Part IV — it is the fastest, safest lever available and cannot strand an in-progress trip.
3. Do not deploy a fix under pressure without diffing it against what's actually live — see the Appendix's "never blindly overwrite a downloaded Edge Function" rule; a rushed redeploy has previously destroyed undocumented production-only work.
4. Capture the exact error code/message and the affected table/function before making any change, so the incident is diagnosable after the fact.
5. Re-enable any kill-switch you flipped as soon as the underlying issue is fixed — these are meant to be temporary.
6. Write down what happened for the Appendix Change Log once resolved.

PART VI

DATABASE REFERENCE

Chapter 54. Complete Table Directory

~55 tables exist across 248 migration files at generation time. This is the full list in first-appearance order; detailed entries for the most operationally important tables follow in this Part.

Table	Purpose
admins	Admin user accounts.
admin_logs	Audit log of admin actions.
dispatch_config	Dispatch/matching tuning configuration.
distance_discounts	Distance-based fare discount tiers.
users	Passenger/user accounts, linked to `auth.users`.
rider_kyc	Rider KYC documents and status.
riders	Rider/Drop Partner accounts.
pricing_zones	Geo-fenced pricing zones.
pricing_config	Fare pricing parameters per zone/vehicle.
orders	Core ride/parcel order table (local and intercity share this table).
order_assignment_logs	Log of every rider-assignment attempt per order.
order_status_history	Status transition history per order.
payments	Payment records (Cashfree and driver-attested).
pricing_change_logs	Audit log of `pricing_config` changes.
rider_assignments	Rider-to-order assignment records (dispatch offers).
rider_subscription_history	WayDrop MAX subscription purchase/status history.
rider_payout_accounts	Rider bank/UPI payout destination accounts.
rider_payout_updates_log	Audit log of payout account changes.
rider_withdrawals	Rider withdrawal requests.
rider_withdrawal_events	Status-event log per withdrawal.
rider_travel_plans	Intercity rider-published travel-plan corridors.
order_chat_messages	In-order chat messages.
rider_reviews	Ratings/reviews for riders.
intercity_requests	Intercity passenger/sender negotiation requests.
user_devices	User device registrations.
promo_codes	Promo code definitions.
promo_code_users	Per-user promo assignment.

Table	Purpose
user_promo_usages	Promo redemption log.
wallet_transactions	Wallet ledger transactions.
wallet_audit_logs	Wallet audit trail.
parcel_security_logs	Parcel security/compliance event log.
order_declarations	Parcel content declarations.
police_sos_logs	SOS/emergency trigger log.
order_parcel_photos	Parcel photo evidence.
parcel_compliance_reports	Parcel compliance reporting.
rider_daily_activity	Daily rider activity/earnings rollup.
otp_attempts	OTP attempt rate-limiting.
trip_share_tokens	Shareable trip-tracking tokens.
user_device_tokens	FCM push tokens per user.
feature_flags	Dropped/legacy feature-flag table — do not resurrect, see Part II.
notifications	Notification records sent to users/riders.
order_verifications	Order pickup/delivery OTP/photo verification records.
rider_vehicles	Rider vehicle registrations beyond the original KYC vehicle.
travel_matches	Matches between travel plans and intercity requests.
payment_disputes	Payment dispute records.
subscription_plans	WayDrop MAX subscription plan catalog.
tax_rate_config	Government tax rate configuration.
government_tax_config	Government tax module threshold configuration.
government_tax_ledger	Per-rider government tax due ledger.
data_deletion_requests	User-initiated account deletion requests.
rider_kyc_retention_archive	Archived KYC evidence retained after account deletion.
deleted_user_archive	Anonymized archive of deleted user accounts.
cod_settlement_ledger	Cash-on-delivery settlement tracking.
diagnostic_timings	Temporary performance-diagnostic timing table.
production_config	Live feature-flag / emergency-control configuration (current, canonical — see Part II).
notification_failures	Failed-notification tracking (see Part III, Notifications).

Chapter 55. Core Identity & Account Tables

users

- **Purpose:** Passenger account; also the account record when a phone number's owner is only ever a passenger.
- **Primary key:** `id` (uuid) = `auth.users.id`.
- **Important columns:** `phone`, `full_name`, `email`, `profile_completed`, `onboarding_completed`, `preferred_mode`/`last_active_mode` (dual passenger/rider mode), `role` (user/admin/support), `pref_order_notifications`/`pref_chat_notifications`/`pref_rating_notifications`/`pref_admin_notifications`, `order_notification_mode`.
- **Sensitive data:** phone, email.
- **Important triggers:** `prevent_role_self_escalation`.
- **Do NOT manually change:** `role`, other than through an intentional out-of-band admin process.
- **Related Flutter screens:** auth/onboarding flow, `home_tab.dart`.

rider_kyc

- **Purpose:** Rider identity/KYC documents and approval status. Highest-sensitivity table in the schema.
- **Primary key:** `id`; FK `user_id` → `users`.
- **Important columns:** `full_name`, `father_name`, `vehicle_type`, `aadhaar_number` (12-digit, checked), `pan_number` (regex-checked), `driving_license_number`, `vehicle_number`, 10 image columns, `kyc_status` (pending/under_review/approved/rejected/resubmitted), `activation_status` (inactive/profile_pending/active/suspended), `rejection_count`, `blocked_until`, `date_of_birth` (18+ check).
- **Storage:** bucket `rider_kyc`, path `{user_id}/{phone}_attempt_{n}/{fileName}.jpg`.
- **RLS history:** an insecure public-read policy was found and removed (migrations `20260808000007`/`20260808000008`) — treat any wide-open SELECT policy discovered on this table as a regression.
- **Do NOT:** edit approved/rejected rows outside the intended admin workflow; delete rows directly (see `rider_kyc_retention_archive`).
- **Related Edge Functions:** `create-rider-kyc`, `validate-rider-kyc`, `cleanup-abandoned-kyc-uploads`.

riders

- **Purpose:** The Drop Partner account (there is no separate `partners` table).
- **Primary key:** `id`; FK `kyc_id` → `rider_kyc`, `user_id` → `users`.
- **Important columns:** `rating` (Bayesian-averaged, floor 3.5), `total_orders`/`total_completed_orders`/`total_cancelled_orders`, `total_earnings`, `today_earnings`, `wallet_balance`, `cod_pending_amount`, `cod_limit` (default 500), `account_status` (online/offline/busy/blocked), `current_lat`/`current_lng`/`current_heading`, `last_location_updated_at`, `subscription_active`, `subscription_expires_at`, `commission_waiver_active`, `is_verified`, `government_tax_due_balance`.
- **Important triggers:** `trigger_assign_nearby_orders` (dispatch-on-location-update), `protect_rider_financial_columns_trigger`, `protect_rider_upi_verification_trigger`, `trigger_prevent_service_mode_switch_during_active_order`, `trigger_enforce_rider_assignment_service_mode`.

- **Do NOT manually change:** `wallet_balance`, `total_earnings`, `cod_pending_amount`, `government_tax_due_balance` — all protected by trigger and/or only correctly mutated by SECURITY DEFINER functions.
- **Related Edge Functions:** `accept-order`, `create-rider-withdrawal`, `create-rider-payout`.

rider_vehicles

- **Purpose:** Additional vehicles beyond a rider's original KYC vehicle.
- **Primary key:** `id`; FK `rider_id` → `riders`.
- **Important columns:** `vehicle_type`, `vehicle_number`, `rc_front_image`, `rc_back_image`, `vehicle_image`, `status` (pending/approved/rejected), `rejection_reason`.
- **RLS:** rider can SELECT/INSERT own vehicles only; approval has no client-facing UPDATE policy (admin/service-role-only, mirroring KYC).

Chapter 56. Order & Dispatch Tables

orders

- **Purpose:** Core ride/parcel order table for both local and intercity flows.
- **Primary key:** `id`; `order_code` (human-readable, auto-generated); FK `user_id` → `users`, `rider_id` → `riders` (nullable until accepted), `pricing_config_id` → `pricing_config`, `payment_id` → `payments`.
- **order_type:** ride, intercity_ride, ride_sharing, local_parcel, intercity_parcel, plus batch_delivery/food_delivery/grocery_delivery (unused placeholders — verify before assuming active).
- **order_status lifecycle:** see Part III, Active Rides & Order Lifecycle, for the exact per-type transition table.
- **Fare columns:** `actual_fare`, `current_fare`, `final_fare`, `base_fare`, `distance_fare`, `time_fare`, `waiting_charge`, `platform_fee`, `gst_percentage`/`gst_amount`, `commission_percentage`/`commission_amount`, `rider_earning`, `boost_amount`/`boost_company_earning`/`boost_rider_earning`/`boost_percentage`, `subscription_commission_waived`, `pricing_snapshot` (jsonb).
- **Dispatch-state columns:** `search_expires_at`, `dispatch_attempts`, `current_search_radius_km`, `dispatch_worker_running`/`dispatch_worker_started_at`, `rider_rejections_count`, `redispatch_count`.
- **Parcel-specific:** `parcel_type`, `receiver_name`/`phone`, `weight_category`, `actual_weight_kg`, `parcel_images[]`, `parcel_declared_value`, `high_risk_parcel`.
- **Compliance:** `sender_declaration_accepted`, `pickup_verification_completed`, `compliance_flagged`/`compliance_notes`/`compliance_status`/`compliance_score`.
- **Important triggers:** `trigger_order_status_update` (feeds the notification pipeline), `protect_order_financial_columns_trigger`, `protect_order_state_columns_trigger`, `trg_populate_order_code`.
- **Do NOT manually change:** `order_status` (use the proper Edge Function path), any fare/commission column.
- **Related Edge Functions:** `create-order`, `accept-order`, `update-order-status`, `boost-order-fare`, `cancel-order`, `reject-order`.

rider_assignments / order_assignment_logs

- **Purpose:** `rider_assignments` is the live dispatch-offer record per rider per order (sent/accepted/rejected/expired); `order_assignment_logs` is the permanent log of every attempt.
- **Related functions:** `find_nearby_riders`, `assign_nearby_searching_orders_to_rider`, `redispatch_freed_riders`.

order_status_history

- **Purpose:** Append-only log of every status transition, with `changed_by` (required — see the audit-trail-gap warning in Part III).

Chapter 57. Intercity Tables

rider_travel_plans

- **Purpose:** A traveller's published intercity corridor.
- **Primary key:** `id`; FK `rider_id` → `riders`.
- **Important columns:** `pickup_city`/`destination_city`, `pickup_lat`/`lng`, `destination_lat`/`lng`, `weight_preference`, `travel_date`, `status` (active/completed/cancelled). Note: `created_at`/`updated_at` are stored as TEXT rather than `timestamptz` on this table — a schema quirk to be aware of if writing new queries that sort/compare these columns.
- **Important triggers:** `trigger_reject_past_travel_plan_date`, `trigger_on_rider_travel_plan_insert`, `trigger_guard_travel_plan_cancellation`, `trigger_cascade_cancel_travel_plan_dependents`, `prevent_delete_active_travel_plan`.

intercity_requests

- **Purpose:** The intercity negotiation/request record — this IS the "intercity order" concept; there is no separate `intercity_orders` table.
- **Primary key:** `id`; FK `parcel_id` → `orders`, `route_id` → `rider_travel_plans`, `traveller_id` → `riders`, `sender_id` → `users`.
- **Important columns:** `initiated_by` (traveller/sender), `status` (request_sent/discussion_started/confirmed/paid/in_progress/completed/cancelled — a distinct status machine from `orders.order_status`), `proposed_fare`, `sender_confirmed`/`traveller_confirmed`.
- **RLS:** participants only (sender or traveller).

travel_matches

- **Purpose:** Idempotency ledger between travel plans and orders, `UNIQUE(order_id, plan_id)` — guarantees a genuinely new match only ever notifies once.

Chapter 58. Payments & Financial Tables

payments

- **Purpose:** Every payment record — order fares, subscriptions, tax-due repayment, COD settlement.
- **Primary key:** `id`; `payment_code` (auto-generated); FK `user_id`/`rider_id` (nullable), `plan_id` → `subscription_plans`, `reference_id` (polymorphic depending on `payment_for`).
- **Important columns:** `amount`, `gst_amount`, `platform_fee`, `discount_amount`, `final_amount`, `payment_method`, `payment_gateway`, `gateway_transaction_id`/`gateway_order_id`,

`payment_status` (pending/paid/failed/refunded/partial_refund/cash_collected/cash_pending_settlement), `paid_at`, `webhook_received`, `webhook_payload` (jsonb — treat as sensitive), `rider_earning`/`company_earning`, `failure_reason`.

- **Do NOT manually change:** `payment_status`, `amount`, `final_amount`, `webhook_received`.
- **Related Edge Functions:** `create-payment`, `cashfree-webhook`, `verify-payment`, `create-fare-qr`.

subscription_plans

- See the full reference and price-change recipe in Part IV, Changing Subscription Prices.
- **Primary key:** `plan_id` (text). **RLS:** `authenticated` may SELECT active rows only; no client INSERT/UPDATE/DELETE policy exists for anyone but `service_role` — the documented supported price-change path is a plain UPDATE via the Dashboard/SQL editor.

rider_subscription_history

- **Purpose:** Subscription purchase/status history — there is no separate "rider_subscriptions" table.
- **Important columns:** `amount` (price snapshot at purchase time), `payment_status`, `starts_at`, `expires_at`, `is_active`.
- **Unique index:** `idx_unique_subscription_payment` on `payment_id` — the idempotency guarantee behind the entitlement-recovery recipe in Part V.

government_tax_config / tax_rate_config / government_tax_ledger

See Part III, Government Tax, for full behavior. `government_tax_ledger` is append-only; unique partial indexes guarantee exactly one `ORDER_TAX` per order and at most one `TAX_PAYMENT` per `payment_id`.

cod_settlement_ledger / wallet_transactions / wallet_audit_logs

Tracks cash-on-delivery settlement and general wallet ledger movements/audit trail respectively. `prevent_wallet_transaction_modifications` and `audit_rider_wallets` are the relevant guard/audit functions — wallet history rows, like the government tax ledger, are not meant to be hand-edited.

Chapter 59. Payout & Withdrawal Tables

rider_payout_accounts / rider_payout_updates_log

- **Purpose:** A rider's registered bank/UPI payout destination (`rider_payout_accounts`) and a rate-limiting audit log of changes to it (`rider_payout_updates_log`).
- **Important columns (rider_payout_accounts):** `beneficiary_id` (auto-generated), `beneficiary_name`/`email`/`phone`, `account_number`, `ifsc_code`, `upi_vpa`, `cashfree_beneficiary_added`, `cashfree_beneficiary_status`, `payout_method` (bank_account/upi).
- **Rate limit:** `check_payout_update_limit` blocks more than 2 field changes in a rolling 7-day window.
- **Sensitive data:** `account_number`, `ifsc_code`, `upi_vpa`, contact details — never expose these in an admin export or log.

rider_withdrawals / rider_withdrawal_events

See the full mechanism in Part III, Payouts & Withdrawals. All writes go exclusively through `initiate_withdrawal`/`fail_withdrawal`/`complete_withdrawal` — no client-facing INSERT/UPDATE/DELETE policy exists on either table.

Chapter 60. Notification Tables

notifications

- **Purpose:** Every notification record sent to a user/rider.
- **Important columns:** `title`, `body`, `notification_type`, `reference_id`/`reference_type`, `is_read`, `push_sent`, `delivery_status` (default `'pending'`), `delivery_error`, `payload` (jsonb).
- **RLS:** users can SELECT/UPDATE their own notifications only.
- **Do NOT:** bulk-modify historical rows; do not assume a `'pending'` row will self-resolve — this was exactly the bug `notification_failures` now surfaces.

notification_failures

See the full mechanism in Part III, Notifications. Admin-only SELECT; no write policy for anyone but `service_role`; no retry mechanism by design.

Chapter 61. Ratings, Config & Supporting Tables

rider_reviews

See Part III, Ratings and Feedback, for full behavior including the Bayesian rating-recalculation trigger. Immutable after INSERT for every role.

production_config, dispatch_config, pricing_config, pricing_zones

`production_config` is fully documented in Part II. `dispatch_config` holds round-escalation radius/timeout tuning for local dispatch (code-verified as real and actively read by `find_nearby_riders`/`start-dispatch`, per `nearby_partner_count_regression_test.dart`) — its exact live numeric values were NOT independently confirmed in this pass (no live DB access) and one of the project's own docs contains two internally-conflicting value sets for the same profile, so treat any specific radius/timeout number as NOT VERIFIED IN CURRENT PROJECT until confirmed via a direct SELECT. `pricing_zones`/`pricing_config` define geo-fenced fare rules per vehicle type; changing them is a data operation (INSERT/UPDATE), not a migration, but should be tested via `calculate-order-pricing`'s fare preview before assuming a new zone/config is correctly picked up.

READ-ONLY

```
SELECT * FROM public.dispatch_config;

SELECT * FROM public.pricing_zones WHERE is_active = true;
```

promo_codes / promo_code_users / user_promo_usages

Promotional discount codes, per-user eligibility, and redemption history respectively. `validate_and_calculate_promo` and `handle_promo_usage_changes` are the relevant functions.

Compliance & safety tables

`order_declarations` (parcel content declarations), `order_parcel_photos` (photo evidence, written by `process-verification-photo`/`bypass-verification-photo`), `parcel_security_logs` (security/compliance event log), `parcel_compliance_reports`, `police_sos_logs` (SOS/emergency trigger log), `otp_attempts` (rate-limiting), `trip_share_tokens` (shareable trip-tracking links).

Account lifecycle tables

`data_deletion_requests`, `deleted_user_archive` (anonymized archive), `rider_kyc_retention_archive` (KYC evidence preserved after account deletion for compliance — see Part III, Account Deletion).

PART VII

FUNCTION & EDGE FUNCTION REFERENCE

Chapter 62. SQL Function Directory

118 unique SQL functions exist across migrations. Grouped by area:

Category	Functions
Dispatch/matching	find_nearby_riders, assign_nearby_searching_orders_to_rider, assign_nearby_searching_orders_core, redispatch_freed_riders, redispatch_stalled_searching_orders, rider_self_rematch, expire_stale_searching_orders, expire_stale_active_orders, prevent_local_ride_assignment_to_busy_rider
Order lifecycle/codes	generate_unique_order_code, complete_order_and_settle, credit_rider_for_completed_order, protect_order_financial_columns, protect_order_state_columns, calculate_fare_breakdown, generate_intercity_delivery_otp_after_proof
Intercity	accept_intercity_offer, accept_intercity_request, cancel_intercity_request_by_traveller, cancel_travel_plan_by_traveller, cascade_cancel_travel_plan_dependents, on_intercity_request_change, on_rider_travel_plan_insert, prevent_request_on_full_travel_plan, sync_travel_plan_status_after
Notifications	send_push_notification_fnc, deliver_push_notification_http_fnc, on_order_status_update, send_order_status_notifications_core, on_notification_insert, on_new_chat_message, on_rider_assignment_change, on_rider_review_insert
Payments/subscriptions/tax/wallet	apply_cod_settlement_payment, apply_government_tax_payment, apply_order_government_tax, is_rider_blocked_by_tax_due, is_rider_blocked_by_cod_limit, activate_rider_free_trial, protect_rider_upi_verification, audit_rider_wallets, prevent_wallet_transaction_modifications, validate_and_calculate_promo
Withdrawals/payouts	initiate_withdrawal, fail_withdrawal, complete_withdrawal, generate_rider_beneficiary_id, check_payout_update_limit, log_rider_withdrawal_event
Security/RLS helpers	check_is_order_passenger, check_is_order_candidate_rider, prevent_role_self_escalation, enforce_order_rider_service_mode, enforce_rider_assignment_service_mode, protect_rider_financial_columns, rider_is_on_active_trip

Category	Functions
Account lifecycle	delete_own_account, delete_user_account, set_compliance_retention, trigger_cleanup_abandoned_kyc_uploads
OTP/misc	check_otp_lockout, log_otp_attempt, expire_trip_sharing_tokens, get_trip_share_token, generate_unique_payment_code/plan_code/transfer_id/user_code, get_dispatch_round_status, get_rider_reliability_stats, update_rider_average_rating, update_user_average_rating

Chapter 63. Trigger Directory

46 unique triggers exist. The generic `set_updated_at` is reused across most tables with an `updated_at` column. Business-significant triggers by table:

Table	Triggers
riders	trigger_assign_nearby_orders, trigger_assign_nearby_orders_on_rider_freed, protect_rider_financial_columns_trigger, protect_rider_upi_verification_trigger, trigger_prevent_service_mode_switch_during_active_order
orders	trigger_order_status_update, protect_order_financial_columns_trigger, protect_order_state_columns_trigger, trg_populate_order_code, trg_orders_compliance_retention
rider_reviews	tr_update_rider_average_rating, trigger_rider_review_insert
rider_travel_plans	trigger_reject_past_travel_plan_date, trigger_on_rider_travel_plan_insert, trigger_cascade_cancel_travel_plan_dependents, trigger_guard_travel_plan_cancellation, prevent_delete_active_travel_plan
intercity_requests	trigger_intercity_request_change
wallet_transactions	trigger_prevent_wallet_transaction_mods
users	trigger_prevent_role_self_escalation, tr_update_user_average_rating
rider_withdrawals	log_withdrawal_event
rider_payout_accounts	trigger_generate_beneficiary_id, trigger_check_payout_update_limit

Chapter 64. Edge Function Directory Overview

28 Deno Edge Functions exist under `supabase/functions/`, no Node tooling. Deployed function **versions** could not be verified against live production in this pass (no `supabase` CLI/credentials available) — everything below reflects repository source only.

Function	One-line purpose
create-order	Central order-creation entry point for all order types.
accept-order	Rider finalizes acceptance of a local (non-intercity) order.

Function	One-line purpose
accept-intercity-request	Rider (traveller) accepts an intercity negotiation.
update-order-status	Central lifecycle state machine for everything after acceptance.
start-dispatch	Local ride/parcel matching/dispatch loop.
match-intercity-riders	Intercity discovery-only matching engine.
create-payment	Central Cashfree payment-session creation for all payment types.
cashfree-webhook	Cashfree's authoritative server-to-server payment-status callback.
verify-payment	Client-triggered poll mirroring the webhook's success handling.
create-fare-qr	Driver-facing Cashfree hosted-checkout "Scan and Pay" QR.
boost-order-fare	Passenger adds extra ■ to a searching order.
send-push-notification	The single outbound FCM delivery point.
regenerate-otp	Re-issues a pickup/delivery OTP.
create-rider-kyc	Rider KYC submission/resubmission.
validate-rider-kyc	Read-only pre-flight KYC validation.
process-verification-photo	Primary pickup/dropoff photo verification.
bypass-verification-photo	Audited fallback to skip photo verification.
create-rider-payout	Registers/updates a rider's Cashfree payout beneficiary.
create-rider-withdrawal	Requests a wallet withdrawal.
delete-account	Server-authoritative account deletion/anonymization.
cleanup-abandoned-kyc-uploads	Scheduled sweep of orphaned KYC uploads.
cancel-order	Thin, JWT-forwarding proxy to update-order-status(cancelled).
reject-order	Rider declines an offered local order.
calculate-order-pricing	The single fare-calculation engine.
get-live-order-status	Polled live order+rider+ETA endpoint for tracking screens.
sms-hook	Supabase Auth SMS hook, routes OTP through MSG91.
get-msg91-balance	Ops utility — remaining SMS credit balance.
load-testing	Synthetic load-generation harness (must stay disabled in production).

Chapter 65. create-order

Purpose: Single central order-creation entry point for every order type. **Auth:** JWT bearer; enforces `body.user_id === caller`. **Tables written:** `orders`, `user_devices`, `user_promo_usages`, `order_status_history`, `parcel_security_logs`, `order_declarations`. **Calls:** `calculate-order-pricing` (authoritative server-side fare recompute), then fire-and-forget `start-dispatch` or `match-intercity-riders`.

Feature flags: `local_rides_enabled`+`ride_booking_enabled` / `parcel_enabled` / `intercity_enabled` depending on order type — returns 503 `FEATURE_DISABLED`. **Idempotency:** client-supplied `idempotency_key`, backed by a partial unique index; a race returns the winning row instead of erroring. **Safety guards:** Haversine floor-clamp and 5x-straight-line ceiling clamp on client-reported distance; a non-cash payment_id is independently verified against `payments` before the order is marked paid. **Troubleshooting symptoms:** 503 → feature flag; fare looks wrong → check `calculate-order-pricing`, not this function; duplicate orders → confirm the client actually sent a stable `idempotency_key`.

Chapter 66. accept-order

Purpose: Rider finalizes acceptance of a local (non-intercity) order. **Auth:** JWT + rider ownership. **Tables:** reads `orders`/`riders`/`rider_assignments`; writes `rider_assignments`, `order_assignment_logs`, `orders`. **Business rule:** WayDrop MAX active subscription zeroes the effective commission percentage in the fare-breakdown recompute (a previously-fixed bug fed the driver's own earnings back into this recompute, silently collapsing the passenger's total — now uses `order.current_fare`). **Concurrency:** optimistic lock (`WHERE order_status='searching'`) → 409 if already accepted. **Structured error codes:** `WRONG_SERVICE_MODE`, `TAX_DUE_THRESHOLD_BLOCKED`, `COD_LIMIT_EXCEEDED`. **Troubleshooting symptoms:** a rider's accept silently fails → read the exact error `code`, don't assume a generic bug.

Chapter 67. accept-intercity-request

Purpose: Rider (traveller) accepts an intercity negotiation — a thin wrapper; nearly all logic lives in the DB RPC `accept_intercity_request`. **Feature flag:** `intercity_enabled` only. **Idempotent:** `ORDER_ALREADY_ASSIGNED` guard on an already non-searching order.

Chapter 68. update-order-status

Purpose: The central lifecycle state machine for everything after acceptance. **Deliberately NOT gated** by any feature flag, so an emergency module disable can never strand an in-progress trip. **Tables:** `orders`, `rider_assignments`, `order_assignment_logs`, `rider_travel_plans`, `intercity_requests`, `order_status_history`. **Key RPC:** `complete_order_and_settle` (atomic completion, replacing what used to be up to 7 sequential round trips). **Payment-completion guard:** a Cashfree-tracked order can only complete once `payment_status='paid'` (webhook-set); a driver-collected order can only be marked paid by the assigned driver's own attestation — passenger self-completion is blocked outright. **Concurrency:** optimistic lock on non-completion transitions. **Troubleshooting symptoms:** 409 CONFLICT → someone else already transitioned it, re-fetch current state; a passenger unable to "complete" a cash order themselves is expected behavior, not a bug.

Chapter 69. start-dispatch

Purpose: The local (non-intercity) matching/dispatch loop — round-based radius escalation until accepted or expired; intercity orders skip this entirely. **Distributed lock:** `orders.dispatch_worker_running`/`dispatch_worker_started_at`, compare-and-swap; a lock older than 120s is force-acquired. **Self-chaining:** capped at roughly 100 seconds of execution per invocation (Deno edge function limit); re-triggers itself fire-and-forget if still searching near that cap. **Give-up path:** calls `expire_stale_searching_orders()` once `max_search_duration_sec` is exceeded — the same function a periodic backstop also calls. **Troubleshooting symptoms:** an order stuck `searching` with `dispatch_worker_running=true` and a stale `dispatch_worker_started_at` (well past 120s) suggests a crashed worker that hasn't yet been reacquired — it should self-heal on the next trigger.

Chapter 70. match-intercity-riders

See full behavior in Part III, Intercity Ride & Parcel Matching. **Discovery-only** — never auto-creates a negotiation. **Auth:** JWT or trusted service-role (used by create-order's background call). **Known limitation:** matching radii are hardcoded in source, not configurable via any table.

Chapter 71. create-payment

Purpose: Single central entry point for creating any Cashfree payment order (fares, subscriptions, government tax repayment, COD settlement). Production-only Cashfree configuration — no sandbox branch. **Auth:** JWT bearer. **Never trusts client amount:** order-fare amount comes from the server-held `orders` row; tax_due/cod_settlement amount from the server-read balance; subscription amount from `subscription_plans.amount` by `plan_id` — the client's own amount field is ignored entirely. **Vehicle-specific pricing enforcement:** a vehicle-tied subscription plan requires a verified matching vehicle (KYC or `rider_vehicles`), fails closed with 403. **Feature flags:** `payments_enabled` (master), `subscriptions_enabled` (additionally, for subscription payments). **Duplicate-session handling:** live-checks Cashfree's own order status before reuse, fixing a historical "stuck on a dead session forever" bug. **Troubleshooting symptoms:** 503 → a payments/subscriptions flag; a subscription retry stuck on an old dead session is the exact bug this function's duplicate-check already fixes — confirm the deployed version is current if it recurs.

Chapter 72. cashfree-webhook & verify-payment

cashfree-webhook is Cashfree's authoritative server-to-server callback — the "money actually moved" signal. Verified via a timing-safe HMAC-SHA256 signature computed over the RAW unparsed request body (re-serializing the JSON first breaks the signature); rejects with 400 on any header mismatch. **Idempotent:** a payment already `paid` short-circuits (but still self-heals subscription entitlement, since that's a separate step). **Known race:** a late webhook arriving after the driver already closed out the order directly is handled by scoping the order UPDATE to `payment_status='payment_pending'` only — it logs "needs manual review" rather than corrupting an already-settled order.

verify-payment is the client-triggered poll used while waiting for the webhook — functionally a mirror of the webhook's success-handling logic (deliberately kept identical, since either path can be the one that first observes success). **Auth:** JWT + explicit ownership check.

Troubleshooting symptoms (both): signature mismatch → check `CASHFREE_SECRET_KEY` matches between Supabase secrets and Cashfree's dashboard; a payment confirmed on Cashfree but not reflected locally → trigger `verify-payment` from the client, or see the Subscriptions manual-recovery recipe in Part V for the underlying idempotent-entitlement pattern (the same shape applies to order-fare settlement).

Chapter 73. create-fare-qr & boost-order-fare

create-fare-qr powers the driver's "Cashfree QR" button. WayDrop does not self-render a raw UPI QR (Cashfree's dedicated raw-QR API is permanently blocked on this merchant account, `s2s_enabled_not_approved`) — instead it creates a UPI-restricted Cashfree hosted-checkout session, and Cashfree's own checkout UI renders the genuine QR. Never surfaces a raw Cashfree error to the driver; the real diagnostic is persisted to `payments.metadata.cashfree_qr_last_error`.

boost-order-fare lets a passenger add extra ■ to an already-searching order; every financial figure is recomputed server-side via the same `calculate_fare_breakdown` RPC as `accept-order`. Hardcoded `MAX_FARE_CEILING = 99999` sanity cap. Restarts the search-timeout clock and re-triggers

`start-dispatch` with the new fare.

Chapter 74. send-push-notification

Purpose: The single outbound FCM delivery point — every notification-triggering DB trigger funnels through here via `pg_net`. Does not create the `notifications` row itself (already inserted inside the triggering transaction). **Auth:** dual-mode — the service-role key is trusted (may target any user); an anon-key or user JWT may only notify itself. **Feature flag:** `notifications_enabled` — when disabled, returns `{success:false, skipped:true}` without touching the already-inserted `notifications` row. **Per-user preference gating:** reads `users.pref_*` and `order_notification_mode`. **Self-cleaning:** auto-deletes a `user_device_tokens` row on an FCM UNREGISTERED/404/410 response.

Chapter 75. regenerate-otp

Purpose: Re-issues a pickup or delivery OTP. Identity is resolved strictly from the caller's own JWT, never the request body. **Order-state gating:** pickup OTP only while accepted/rider_arrived/rider_at_pickup; delivery OTP only while picked_up/rider_at_drop. **Writes:** new OTP + resets attempt counter; logs to `order_status_history` with `metadata.regenerated:true`.

Chapter 76. create-rider-kyc, validate-rider-kyc, process-verification-photo, bypass-verification-photo

create-rider-kyc: full KYC submission/resubmission; strict ownership check (`body.user_id === caller` — the only real gate, since the function uses a service-role client that bypasses RLS); age gate (18+); resubmission cooldown (7-day block after 2 rejections); stores bare storage paths, never full URLs. **validate-rider-kyc:** read-only pre-flight check, same validation rules, no writes.

process-verification-photo: the primary (non-bypass) pickup/dropoff photo verification — downloads and SHA-256-hashes the uploaded photo server-side for tamper evidence; an SSRF guard requires the photo URL to already be under this project's own Storage path.

bypass-verification-photo: lets the assigned rider skip mandatory photo verification with a typed reason (5+ characters) when the primary path can't be used. This is not a security bypass of auth/ownership — it requires the identical JWT + ownership check as every other order-mutation function, and every use is permanently audit-logged (`order_verifications.verification_status='bypassed'`).

> **WARNING — Provenance note on bypass-verification-photo**
> This function's own source code carries a comment stating it was downloaded read-only from a live production deployment during an audit — it had been deployed directly to production without ever being committed to this repository. Committing it was a version-control fix only, not a behavior change, and it has not been redeployed since. This means the file's git history does not reflect its real deployment history — confirm what is actually live matches this file before assuming a future `supabase functions deploy` of it is a no-op.

Chapter 77. create-rider-payout & create-rider-withdrawal

See the full mechanism in Part III, Payouts & Withdrawals. Both call the Cashfree Payouts V2 API with an RSA-OAEP signed `X-Cf-Signature` header, and both fail fast with a clear error if the signing secret is missing rather than silently omitting the header. `create-rider-withdrawal`'s GET path syncs status from

Cashfree read-only (never deducts the wallet); only the POST path deducts, and only via the atomic `initiate_withdrawal` RPC.

Chapter 78. delete-account & cleanup-abandoned-kyc-uploads

delete-account: see full behavior in Part III, Account Deletion. Identity strictly from the caller's own JWT; KYC evidence archived (never deleted); `auth.users` row banned (~100 years) rather than hard-deleted; blocked (409) while an active order exists.

cleanup-abandoned-kyc-uploads: hard-restricted to the service-role key only — must never be invoked from the Flutter client. Sweeps unclaimed KYC-upload session folders older than a 7-day grace period, and prunes superseded-generation objects within a claimed folder after 1 hour. Paginates Storage listing fully (a documented past bug: unpaginated listing silently stopped scanning past the first 1000 entries).

Chapter 79. Other Edge Functions

cancel-order: a thin proxy to `update-order-status(new_status:"cancelled")`, forwarding the caller's own JWT (not service-role) so the real ownership/transition checks run exactly as if called directly — created because several Flutter screens previously called a nonexistent `cancel-order` function and silently fell back to an unvalidated raw client-side update on 404.

reject-order: a rider declining an offered order — only ever transitions `rider_assignments`, never `orders` itself; sets a 4-minute cooldown (`hidden_until`) before that rider can be re-offered the same order (unless the fare is later boosted above what they last saw).

calculate-order-pricing: the single fare-calculation engine, used both for live client-side fare previews and server-to-server by `create-order`. Resolves a pricing zone (geofence → city-name → substring match) and a vehicle-specific pricing config, applying the same weight/vehicle business rules `create-order` will later enforce so the client never sees a fare for a combination that would be rejected. Supports a "Developer Test Mode" bypass of the zone requirement for developer/admin accounts.

get-live-order-status: polled roughly every 5 seconds by the passenger's live-tracking screen; resolves order+rider+KYC+profile in one embedded-FK query (a documented perf fix, replacing 3 separate queries) plus a fresh signed photo URL and an Ola Maps ETA/distance lookup with short-TTL caching and graceful degradation (`tracking.tracking_degraded:true`) rather than showing a false zero on a transient Maps failure.

sms-hook: Supabase Auth's "Send SMS" hook, routing OTP delivery through MSG91 instead of Supabase's built-in provider. Verified via HMAC webhook signature, not a JWT. Never logs the real phone number or the live OTP value in plaintext.

get-msg91-balance: an internal ops utility reporting remaining SMS credit balance — not designed for end-user exposure.

load-testing: a synthetic load-generation harness, explicitly not a production business function. Hard-disabled unless `LOAD_TEST_ENABLED=true` (should never be true in production) and a shared-secret header. Every mock entity is tagged with a sentinel UUID so it can never touch real data even if accidentally invoked.

> **WARNING**
> Confirm `LOAD_TEST_ENABLED` is unset/false in the production Supabase project's function secrets. The function existing and being deployed is not itself a risk as long as that flag is off.

PART VIII

EMERGENCY PLAYBOOK

Chapter 80. Feature Kill-Switch Quick Reference

Full recipes are in Part IV, Feature Kill-Switches. This page is the one-glance version for an active incident.

Symptom / goal	Flip this `production_config` column	Strands active work?
Take the whole app down for maintenance	`maintenance_mode = true`	No
Block old app versions immediately	`force_update = true` or raise `minimum_android_version`	No
Stop new local ride bookings	`local_rides_enabled = false` and/or `ride_booking_enabled = false`	No
Stop new local parcel bookings	`parcel_enabled = false`	No
Stop new intercity bookings/matches	`intercity_enabled = false`	No
Stop all new payment sessions	`payments_enabled = false`	No
Stop new subscription purchases only	`subscriptions_enabled = false`	No
Stop outbound push notifications	`notifications_enabled = false`	No (in-app inbox unaffected)

SAFE

Every row above is a new-action gate only. Nothing in this table ever touches `update-order-status`, so an order already `accepted` or later always runs to completion regardless of any flag above.

READ-ONLY

```
SELECT * FROM public.production_config WHERE is_active = true;
```

Chapter 81. Maintenance Mode & Force Update

See Part IV for the exact UPDATE statements. Remember: neither of these has a live-push mechanism to an already-open app session — both take effect on the next cold start only.

Chapter 82. Incident Response Checklist

1. Confirm scope: one user, or systemic? (Part I triage funnel.)
2. If systemic and a specific module is implicated, flip the narrowest kill-switch that addresses it — do not disable more than necessary (e.g. disabling `payments_enabled` also blocks subscriptions and tax repayment, not just ride-fare payments).
3. Capture the exact error code/message and affected table/function before changing anything.
4. If a code fix is required under time pressure, diff the current live Edge Function against the repository version first — never blindly redeploy from git HEAD without confirming HEAD actually matches what's live (see the Appendix's "do not blindly overwrite production" rule; this has caused real, permanent loss

of undocumented production-only work before).

5. Communicate the flag change and expected duration if it's user-visible (maintenance mode, force update).
6. Re-enable every kill-switch you flipped as soon as the underlying issue is resolved.
7. Record what happened, when, and why in the Appendix Change Log.

Chapter 83. Rollback Guidance

- **A bad `production_config` change** is the cheapest possible rollback — flip the same column back. No deploy, no migration.
- **A bad Edge Function deploy** — redeploy the previous known-good version. Confirm what "previous known-good" actually means by diffing against what was truly live (see the Change Log for the specific historical incident where this went wrong).
- **A bad migration** — migrations in this project are strictly forward-only; never edit or delete a past migration. A correction is always a new, additive migration that fixes forward.
- **A bad manual data change** — restore via the specific SECURITY DEFINER function's inverse operation where one exists (e.g. `fail_withdrawal` refunds a wallet), never via a second raw UPDATE guessing at the original value.

PART IX

APP / SUPABASE CROSS-REFERENCE

Chapter 84. Flutter Screen → Supabase Object Map

This codebase does not use a `*Controller` naming convention for local_rides/local_parcels/intercity — screens and Riverpod providers call Supabase/Edge Functions fairly directly. Primary screen files by flow:

Flow	Primary screens
Local ride	`lib/features/local_rides/booking/screens/ride_location_screen.dart`, `.../searching/screens/ride_searching_screen.dart`, `.../tracking/screens/ride_tracking_screen.dart`
Local parcel	`lib/features/local_parcels/booking/screens/parcel_booking_screen.dart`, `.../searching/screens/parcel_searching_screen.dart`, `.../tracking/screens/parcel_tracking_screen.dart`
Intercity ride	`lib/features/intercity/rides/booking/intercity_ride_booking_screen.dart`, `.../shared/screens/intercity_searching_screen.dart`, `.../shared/screens/intercity_tracking_screen.dart`
Intercity parcel	`lib/features/intercity/parcels/booking/intercity_booking_screen.dart`, `.../dashboard/intercity_parcels_screen.dart`
Travel plans (traveller)	`lib/features/rider/travel_plans/screens/post_travel_plan_screen.dart` (invokes `match-intercity-riders`), `.../travel_plan_details_screen.dart`
Rider home / earnings / orders	`lib/features/rider/home/rider_home_tab.dart`, `.../earnings/rider_earnings_tab.dart`, `.../orders/rider_orders_screen.dart`, `.../orders/screens/rider_active_order_screen.dart`, `.../orders/widgets/incoming_order_card.dart`
KYC upload	`lib/features/rider/onboarding/screens/rider_kyc_screen.dart` (invokes `validate-rider-kyc` then `create-rider-kyc`), `rider_kyc_upload_service.dart`, `rider_kyc_pending_screen.dart`
Payout / withdrawal	`lib/features/rider/onboarding/screens/rider_payout_details_screen.dart`, `lib/features/rider/payouts/rider_payouts_screen.dart`
Account deletion	`lib/shared/screens/data_deletion_screen.dart` (invokes `delete-account`)
Passenger home (feature-flag gating)	`lib/features/user/home/home_tab.dart`
App start / gating	`lib/app_start_screen.dart`
Maintenance / force-update	`lib/shared/screens/maintenance_screen.dart`, `lib/shared/screens/update_required_screen.dart`
Notifications	`lib/modules/notifications/screens/notifications_inbox_screen.dart`, `notification_detail_screen.dart`, `services/notification_service.dart`

Flow	Primary screens
Payments	`lib/modules/payments/controllers/pricing_controller.dart`, `lib/modules/payments/services/payment_service.dart`

Chapter 85. Flutter Service → Edge Function Map

Flutter service	Edge Function(s) called
`ProductionConfigService` (`lib/core/services/config/production_config_service.dart`)	reads `production_config` directly (not an Edge Function — a direct table SELECT)
`PaymentService` (`lib/modules/payments/services/payment_service.dart`)	`create-payment`, `verify-payment`, `create-fare-qr`
`rider_kyc_upload_service.dart`	`validate-rider-kyc`, `create-rider-kyc`
`intercity_matching_service.dart`	`match-intercity-riders` (via `post_travel_plan_screen.dart`)
`data_deletion_screen.dart`	`delete-account`
`notification_service.dart`	no direct Edge Function call — consumes FCM payloads and `notifications` rows

PRODUCTION

`ProductionConfigService` fetches once per app cold start, caches via `CacheService` (encrypted local cache), and fails OPEN with every flag defaulting to enabled if there is no cache and the fetch fails — the server-side checks in create-order/create-payment/send-push-notification are the real enforcement backstop, not this client cache.

Chapter 86. Table → Function Map

Table	Written by
orders	create-order, accept-order, accept-intercity-request (via RPC), update-order-status, boost-order-fare, cancel-order, regenerate-otp
rider_assignments	start-dispatch, accept-order, reject-order, update-order-status
payments	create-payment, cashfree-webhook, verify-payment, create-fare-qr
rider_kyc	create-rider-kyc, cleanup-abandoned-kyc-uploads (Storage only), delete-account (archival)
rider_withdrawals	create-rider-withdrawal (via initiate_withdrawal/fail_withdrawal/complete_withdrawal RPCs)
notifications	inserted by DB triggers (on_order_status_update, on_new_chat_message, on_rider_review_insert, on_intercity_request_change), delivery status updated by send-push-notification
notification_failures	deliver_push_notification_http_fnc, on_order_status_update, send_order_status_notifications_core, send_push_notification_fnc (failure branches only)

Table	Written by
government_tax_ledger	apply_order_government_tax, apply_government_tax_payment (via update-order-status / create-payment / cashfree-webhook / verify-payment)
rider_subscription_history	cashfree-webhook, verify-payment (applySubscriptionEntitlement)

Chapter 87. Notification Type → Producer → Recipient

Notification type	Produced by	Recipient
order_status	`on_order_status_update` trigger on `orders`	passenger and/or rider depending on transition
chat_message	`on_new_chat_message` trigger on `order_chat_messages`	the other participant in the order chat
intercity_request (travel_partners_available / new_travelers_available / per-match)	`match-intercity-riders` (background mode), `on_intercity_request_change` trigger	passenger and/or traveller
rating/review	`on_rider_review_insert` trigger on `rider_reviews`	rider
kyc_status	admin action on `rider_kyc` (manual, Dashboard-driven today)	rider

WARNING

`on_intercity_request_change` and `on_rider_review_insert` are the two trigger paths NOT covered by the `notification_failures` safety net (see Part III, Notifications) — a silent notification failure originating from these two specifically will not appear in that table.

Chapter 88. Payment Flow → Tables → Functions

```
Passenger/rider requests payment

  -> create-payment (validates amount server-side, inserts `payments` BEFORE calling Cashfree)

  -> Cashfree hosted checkout or create-fare-qr's "Scan and Pay"

  -> Cashfree confirms via cashfree-webhook (HMAC-verified, authoritative)

       and/or the client polls verify-payment (mirrors webhook success logic)

  -> payments.payment_status = 'paid'

  -> orders.payment_status/fare_payment_method updated (order-fare payments only)

     OR rider_subscription_history row inserted (subscription payments)

     OR government_tax_ledger TAX_PAYMENT entry (tax-due repayment)

     OR cod_settlement_ledger entry (COD settlement)
```

Chapter 89. Subscription Flow → Tables → Functions

```
Rider selects a subscription_plans row (by vehicle_type if applicable)
  -> create-payment (payment_for='subscription', plan_id) — vehicle ownership verified via
     rider_kyc/rider_vehicles if the plan is vehicle-specific
  -> Cashfree checkout -> cashfree-webhook / verify-payment
  -> applySubscriptionEntitlement(): riders.subscription_active/subscription_expires_at updated
     (stacks onto existing future expiry), rider_subscription_history row inserted
     (idempotent on payment_id via a unique index)
```

Chapter 90. KYC Flow → Storage → Tables → Screens

```
rider_kyc_screen.dart (incremental per-document upload to Storage as each is picked)
  -> validate-rider-kyc (pre-flight, no writes)
  -> create-rider-kyc (writes rider_kyc row; bare storage paths, never full URLs)
  -> rider_kyc_pending_screen.dart (waiting state)
  -> [admin manually reviews via Supabase Dashboard: Table Editor + Storage browser — no in-app
      admin screen exists]
  -> admin sets rider_kyc.kyc_status='approved', activation_status='active'
  -> rider can go online
```

Orphaned/abandoned upload sessions are swept by `cleanup-abandoned-kyc-uploads` (scheduled, service-role-only) after a 7-day grace period.

PART X

ADMIN CHEAT SHEETS

Chapter 91. 1-Page Emergency Cheat Sheet

READ-ONLY

```
SELECT * FROM public.production_config WHERE is_active = true;
```

Need to...	Do this
Stop everything	`UPDATE production_config SET maintenance_mode = true WHERE is_active = true;`
Block old app versions	`UPDATE production_config SET force_update = true WHERE is_active = true;`
Stop rides / parcel / intercity	Set the matching `_enabled` column to `false` (Part IV)
Stop payments / subscriptions	`payments_enabled` / `subscriptions_enabled = false`
Stop push notifications	`notifications_enabled = false`
Undo any of the above	Set it back to `true`/`false` as appropriate — takes effect on next app cold start

Chapter 92. Subscription Price Change Cheat Sheet

SAFE

```
SELECT plan_id, amount, is_active FROM public.subscription_plans ORDER BY plan_id;

UPDATE public.subscription_plans SET amount = <new_price> WHERE plan_id = '<plan_id>';
```

No migration, no Edge Function deploy, no APK rebuild. Only affects future purchases. Never touch `plan_id`; deactivate via `is_active`, never delete. Full detail: Part IV.

Chapter 93. KYC Cheat Sheet

1. Find the rider: `SELECT * FROM public.rider_kyc WHERE user_id = '<user_id>';` (or search by `aadhaar_number`/`vehicle_number` if you don't have the user_id).
2. View a document/photo: open the Supabase Storage browser on the `rider_kyc` bucket at path `{user_id}/{phone}_attempt_{n}/{fileName}.jpg` — there is no in-app viewer.
3. Approve: `UPDATE public.rider_kyc SET kyc_status='approved', activation_status='active' WHERE id = '<kyc_id>';` then ensure a `riders` row exists for that user (create one if this is a first-time activation).
4. Reject: `UPDATE public.rider_kyc SET kyc_status='rejected', rejection_reason='<reason>' WHERE id = '<kyc_id>';` — `rejection_count` and the 7-day `blocked_until` cooldown are handled automatically on the rider's next resubmission attempt.
5. Never delete a `rider_kyc` row. Never flip the `rider_kyc` bucket back to public.

WARNING
Only `rider_image.jpg` and `vehicle_image.jpg` are ever shown in the app. The other 8 images are review-only via Storage.

Chapter 94. Active Ride Cheat Sheet

READ-ONLY

```
SELECT id, order_code, order_type, order_status, user_id, rider_id, created_at

FROM public.orders

WHERE order_status NOT IN ('completed','delivered','cancelled','expired')

ORDER BY created_at DESC;
```

Find the rider: join `riders`/`users` on the IDs above. Find the route: `pickup_lat`/`pickup_lng`/`destination_lat`/`destination_lng` on the `orders` row. Timestamps that matter: `created_at`, `accepted_at`, `rider_arrived_at`, `picked_up_at`, `rider_at_drop_at`, `completed_at`, `cancelled_at`.

Chapter 95. Withdrawal & Payment Cheat Sheet

READ-ONLY

```
SELECT * FROM public.rider_withdrawals WHERE rider_id = '<rider_id>' ORDER BY created_at
    DESC;

SELECT * FROM public.rider_withdrawal_events WHERE withdrawal_id = '<withdrawal_id>' ORDER
    BY created_at;

SELECT * FROM public.payments WHERE user_id = '<user_id>' OR rider_id = '<rider_id>' ORDER
    BY created_at DESC;
```

Never manually set a withdrawal's `status` or a payment's `payment_status` — resolve via the SECURITY DEFINER RPCs or by confirming with Cashfree directly. Full recipe: Part V.

Chapter 96. Notification & Feature-Disable Cheat Sheet

READ-ONLY

```
SELECT delivery_status, push_sent, delivery_error FROM public.notifications

WHERE user_id = '<user_id>' ORDER BY created_at DESC LIMIT 10;

SELECT * FROM public.notification_failures WHERE user_id = '<user_id>' ORDER BY created_at
    DESC;
```

Feature disable/enable: see the 1-Page Emergency Cheat Sheet above and the full recipes in Part IV.

PART XI

APPENDICES & CHANGE LOG

Chapter 97. Appendix A — What Requires What (Decision Framework)

Kind of change	Requires
A `production_config` value (any flag, maintenance/force-update/version)	Nothing — a plain UPDATE via Dashboard/SQL editor, effective on next client cold start.
A `subscription_plans`, `pricing_config`, `pricing_zones`, or `dispatch_config` value	Nothing — a plain data UPDATE/INSERT. Test via the relevant fare-preview/matching path afterward.
Adding/changing a column, table, RLS policy, trigger, or SQL function	A new forward-only migration (`supabase migration new`, then `supabase db push`). Never edit a past migration.
Edge Function logic (any file under `supabase/functions/`)	`supabase functions deploy <name>` (or all).
An Edge Function secret (e.g. `CASHFREE_SECRET_KEY`, `FIREBASE_SERVICE_ACCOUNT_KEY`)	`supabase secrets set KEY="value"` — this auto-restarts the function runtime.
Any Dart/Flutter UI or client logic change (screens, services, feature-flag gating call sites, notification tap-routing)	A new app build and store release (Android appbundle / iOS ipa) — see `docs/11_Deployment.md`'s build commands. Takes effect only after the user updates.

WARNING
This decision framework was composed from first principles against the actual mechanisms documented in this handbook, since no single project doc states it directly. If in doubt for a specific change, find that exact object's chapter in Part II/III/VI/VII and follow its own "Do NOT" / callout guidance first.

Chapter 98. Appendix B — Known Project-Specific Safety Rules

These are lessons already established by this project's own history — evidenced by migration comments, regression-test headers, and commit messages, not invented for this handbook.

1. Do not blanket-disable `update-order-status` for an emergency — it deliberately has no feature-flag gate specifically so existing accepted/ongoing rides are never stranded.
2. Do not blindly redeploy a locally-downloaded Edge Function over production, and do not "fix" a bad download by restoring from git HEAD without first confirming HEAD actually matches what's live. A real incident permanently lost undocumented production-only performance work this way until it was manually reconstructed.
3. Do not resurrect or duplicate the old, dropped `feature_flags` table/service — `production_config` is the current, separate, minimal design.
4. Never trust a client-supplied payment amount or subscription duration — a confirmed historical vulnerability exploited exactly this for near-free subscriptions before the fix (server-side plan/price lookup, millisecond-precision expiry).

5. Subscription entitlement activation must be idempotent on the existence of a `rider_subscription_history` row, not on `payment_status` — no automatic retry exists yet for this specific gap (manual recovery recipe: Part V).
6. Never directly UPDATE a rider's financial columns (`wallet_balance`, `total_earnings`, `government_tax_due_balance`, `cod_pending_amount`) — trigger-protected, and only correctly mutated by specific SECURITY DEFINER functions.
7. Any function meant to run with elevated privilege from a client-callable path must be `SECURITY DEFINER` and must read live configuration rather than a hardcoded value — a real bug had a passenger-facing partner count silently return zero because it ran under the caller's own restrictive RLS.
8. Never let a UI error handler discard a specific server rejection message for a feature-flag-disabled action — the 503 message from `payments_enabled`/`subscriptions_enabled` IS the intended user-facing communication.
9. Never leak a raw Cashfree error code/message to a driver or passenger — log it server-side only, always show a fixed non-technical message.
10. Never fabricate or reconstruct a raw UPI QR/payment payload client-side — the retired `upi://pay` deep-link architecture was flagged by payment apps as mule-account-pattern activity on real devices.
11. A rider's cancellation attribution must be server-derived, never client-trusted, and written in the same UPDATE that changes order status.
12. PL/pgSQL triggers must not make external HTTP calls directly — queue via a table (typically `notifications`) and process asynchronously via `pg_net` + an Edge Function.
13. RLS is required on every new table, with explicit role grants alongside the policy — a policy alone is not always sufficient for PostgREST.
14. Preserve every accepted/ongoing ride during any emergency shutdown — feature flags gate new actions only, never an order's existing lifecycle.

Chapter 99. Appendix C — Release Checklist (composed from `docs/11_Deployment.md` and this handbook's own findings)

1. Confirm no destructive/backwards migration is included — migrations are additive-only.
2. `flutter analyze` and `flutter test` both clean.
3. Confirm any new Edge Function secret is set via `supabase secrets set` before the function that needs it is deployed.
4. Deploy migrations first (`supabase db push`), then Edge Functions (`supabase functions deploy`), then release the app build — in that order, so a new client build never calls a not-yet-deployed backend contract.
5. Android: `flutter build appbundle --release --obfuscate --split-debug-info=build/app/outputs/symbols`, needs `android/key.properties` configured.
6. iOS: `flutter build ipa --release --obfuscate --split-debug-info=build/ios/outputs/symbols`, archived via Xcode to App Store Connect.
7. After release, consider raising `production_config.latest_android_version` (non-blocking nudge) rather than immediately forcing an update, unless the release fixes something urgent enough to warrant `force_update`/raising `minimum_android_version`.
8. Update this handbook's Change Log with anything operationally significant.

Chapter 100. Appendix D — Cashfree Phase 7B Deferred Checklist

The following payments-related items were identified during this handbook's research as documented-but-not-yet-resolved, and are called out here so they are not lost:

- **"Item 5"** — whether to extend the `notification_failures` safety net to `on_intercity_request_change` and `on_rider_review_insert` remains a separate, explicitly not-yet-approved decision.
- **"Phase 5 Gap D"** — subscription (and by the same pattern, order-fare) entitlement activation has a manual, not automatic, recovery path for the payment-paid-but-entitlement-not-granted race. A root-cause automatic fix is a separate, not-yet-approved decision.
- Exact live `dispatch_config` round-radius/timeout values were not verified against production in this pass, and one project doc contains two internally-conflicting value sets for the same profile — worth reconciling directly against the live table.
- A stale `docs/10_Admin.md` reference to a `kyc_docs` storage bucket (the real bucket is `rider_kyc`) and a stale `docs/01_Architecture.md` reference to the same bucket name should be corrected so future readers of those docs aren't misled.
- **`docs/13_Business_Logic.md` §4** was found during this handbook's preparation to contain real customer PII (a real name, phone number, and a live Storage URL) presented as a "live production record" example. This should be redacted from that document independent of this handbook.

Chapter 101. Appendix E — Change Log

Sourced from `git log`; commit hashes and messages are verbatim, grouped by theme. This is not exhaustive history — it covers the most operationally significant recent commits found during this handbook's preparation.

Notifications & Realtime

- `1ffd8f1` — fix: correct notification tap routing
- `e1c0f8e` — fix: harden realtime pooling and cache cleanup
- `576b55f` — feat: add notification failure tracking and safe trigger wrapping
- `f137fbb` — perf: gate payment fallback polling on realtime health

Subscriptions

- `c0e9196` — fix: correct subscription expiry trial state
- `5f4c35a` — fix: make subscription entitlement activation retry-safe

Performance

- `a204416` — perf: reconcile approved performance batch
- `97145e9` — perf: suppress duplicate rider orders work during active rides
- `1c889a4` — perf: optimize rider active order hot paths
- `41f3bc6` — perf: optimize rider order and notification hot paths
- `24b3bd2` — Optimize app performance and Supabase usage

Payments architecture

- `ec01955` — Add QR-based direct-payment architecture: uploaded Personal QR, UPI Apps deep link, Cashfree QR
- `344de6d` — WayDrop payment, intercity, chat, tax and production fixes
- `8f37dbf` — Stop masking specific server error messages behind a generic snackbar

KYC

- `200320c` — Redesign Rider KYC upload flow with incremental uploads and reliability fixes
- `bdb8955` — Added rider KYC updates, logo changes, app icon updates, ride screen updates

Production stability batches

- `1b10fe6` — Clean up production release configuration
- `ed24dee` — Fix free trial, intercity pricing and egress issues
- `c165cc2` — Complete intercity and rider flow fixes
- `3aff294` — Production release: security fixes, payout, withdrawal, OTP, navigation and stability improvements
- `01b3596` — Fix rider dispatch, cancel flow and production stability issues
- `69fa855` — Production fixes: dispatch, navigation, notification, cancel flow, audit updates
- `31479b4` — Complete security audit fixes, edge functions, migrations and internal testing preparation

Intercity / UX

- `bf97620` — Fix local ride minimize flow and intercity accept dialog
- `78a99f1` — Fix intercity trip UX realtime sync directions and local ride distance
- `0c67049` — Intercity improvements

Chapter 102. Appendix F — How This Handbook Was Produced

This handbook was generated by directly inspecting the WayDrop repository: all `supabase/migrations/*.sql` files (schema, RLS, triggers, functions), every `supabase/functions/*/index.ts` Edge Function, Flutter admin-relevant source (`lib/core/services/config/`, `lib/app_start_screen.dart`, notification and payment services), the project's hand-maintained `docs/*.md` files, `git log`, and `test/*_regression_test.dart` header comments. No live Supabase production database credentials were available (no service-role key, no `supabase` CLI session) — schema and code structure are verified directly against source; live row values are sourced from migration defaults and explicitly flagged where a Dashboard edit since deploy could have changed them. No production data was read, written, or changed in the course of producing this document.

Alphabetical Index

A

accept-intercity-request	54
accept-order	54

B

bypass-verification-photo	56

C

Cashfree	25
Cashfree webhook	25
cashfree-webhook	25, 55
create-order	21, 54
create-payment	25, 55
create-rider-withdrawal	26, 57

D

delete-account	27, 57
dispatch_config	22, 49
driver_personal_qr bucket	13

E

Edge Functions	13, 18

F

feature kill-switch	10, 58
force_update	10, 28

G

Government Tax	26
government_tax_ledger	26

I

intercity_requests	21, 47

K

KYC	27, 64

M

maintenance_mode	10, 28
match-intercity-riders	21, 55

N

notification_failures	23, 49
notifications	23, 49

O

orders	16, 21, 46

P

parcel_images bucket	13
payments	16, 48
pg_net	13
production_config	9, 19, 28, 61
production_config_cache	18

R

Realtime	13

regenerate-otp	56
rider_kyc	27, 45
rider_kyc bucket	13
rider_payout_accounts	26, 48
rider_reviews	24, 49
rider_subscription_history	24
rider_travel_plans	21, 47
rider_vehicles	27, 46
rider_withdrawals	26, 49
riders	14, 46
RLS	16
S	
send-push-notification	56
start-dispatch	21, 54
Storage Buckets	13
Subscription Prices	34
subscription_plans	9, 24, 34
U	
update-order-status	22, 54
users	14
W	
WayDrop MAX	24